

CIS 6270

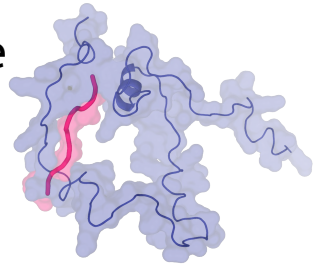
Module 2: Flow Matching

Lecture 2.3/3.1: SDEs, Fokker-Planck, and Score Matching

Tue, Sep 15, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



How do you generate with a trained flow?

How do you generate with a trained flow? Super easy.

After training, discard the paired data endpoint. Begin with a new prior sample:

$$X_0 \sim p_0$$

How do you generate with a trained flow? Super easy.

After training, discard the paired data endpoint. Begin with a new prior sample:

$$X_0 \sim p_0$$

Then solve

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t)}$$

from t = 0 to t = 1.

How do you generate with a trained flow? Super easy.

After training, discard the paired data endpoint. Begin with a new prior sample:

$$X_0 \sim p_0$$

Then solve

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t)}$$

from t = 0 to t = 1.

The terminal state is

$$X_1 = \phi_1(X_0)$$

How do you generate with a trained flow? Super easy.

After training, discard the paired data endpoint. Begin with a new prior sample:

$$X_0 \sim p_0$$

Then solve

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t)}$$

from t = 0 to t = 1.

The terminal state is

$$X_1 = \phi_1(X_0)$$

If the learned field matches the target marginal velocity, $X_1 \sim p_{\text{data}}$

How do you generate with a trained flow? Super easy.

After training, discard the paired data endpoint. Begin with a new prior sample:

$$X_0 \sim p_0$$

Then solve

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t)}$$

from t = 0 to t = 1.

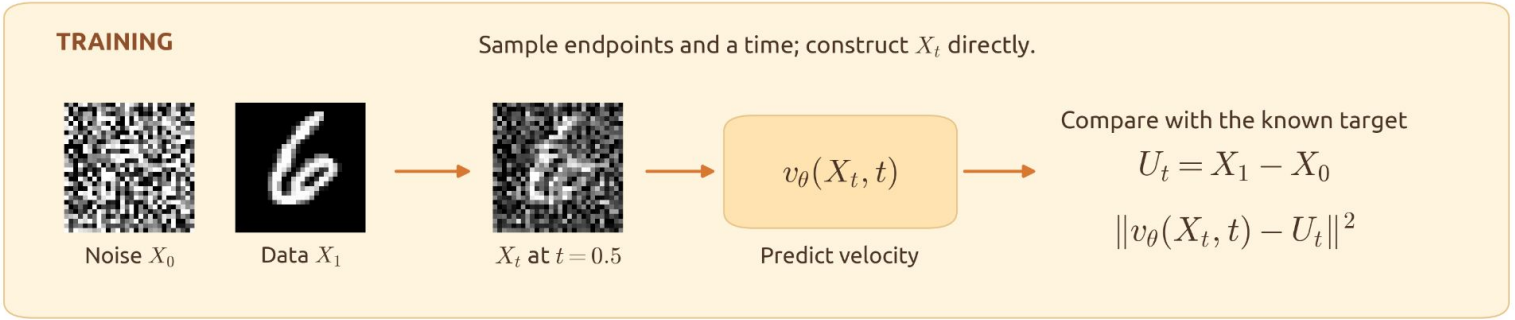
The terminal state is

$$X_1 = \phi_1(X_0)$$

If the learned field matches the target marginal velocity, $X_1 \sim p_{\text{data}}$

Training and generation use the velocity network differently.

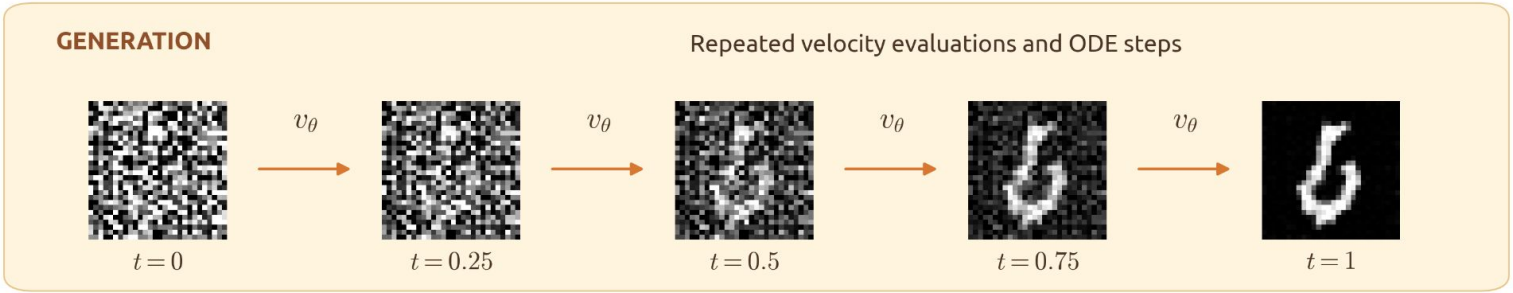
During training, we construct X_t directly from paired endpoints and regress onto a known conditional velocity.



Training and generation use the velocity network differently.

During training, we construct X_t directly from paired endpoints and regress onto a known conditional velocity.

During generation, we have only X_0. We repeatedly evaluate v_θ and numerically integrate the learned ODE until t = 1.



By the way, you can condition the flow while training

By the way, you can condition the flow while training

We can condition the velocity network on c.

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t, c).}$$

By the way, you can condition the flow while training

We can condition the velocity network on c.

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t, c).}$$

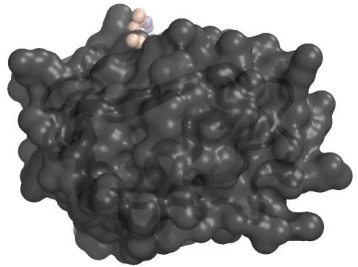
During training, we pair the condition with its corresponding data sample and minimize

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}\left[\left\|v_\theta(X_t, t, c) - U_t\right\|^2\right]$$

By the way, you can condition the flow while training

We can condition the velocity network on c.

$$\boxed{\frac{dX_t}{dt} = v_\theta(X_t, t, c).}$$



During training, we pair the condition with its corresponding data sample and minimize

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}\left[\left\|v_\theta(X_t, t, c) - U_t\right\|^2\right]$$

For molecular generation, c could encode a target protein sequence, a desired molecular class, or measured biological context.

Or on text.

“A picture of a tabby cat”



We’ll go through conditioning at the end of the next set of lectures.

“A picture of a tabby cat”

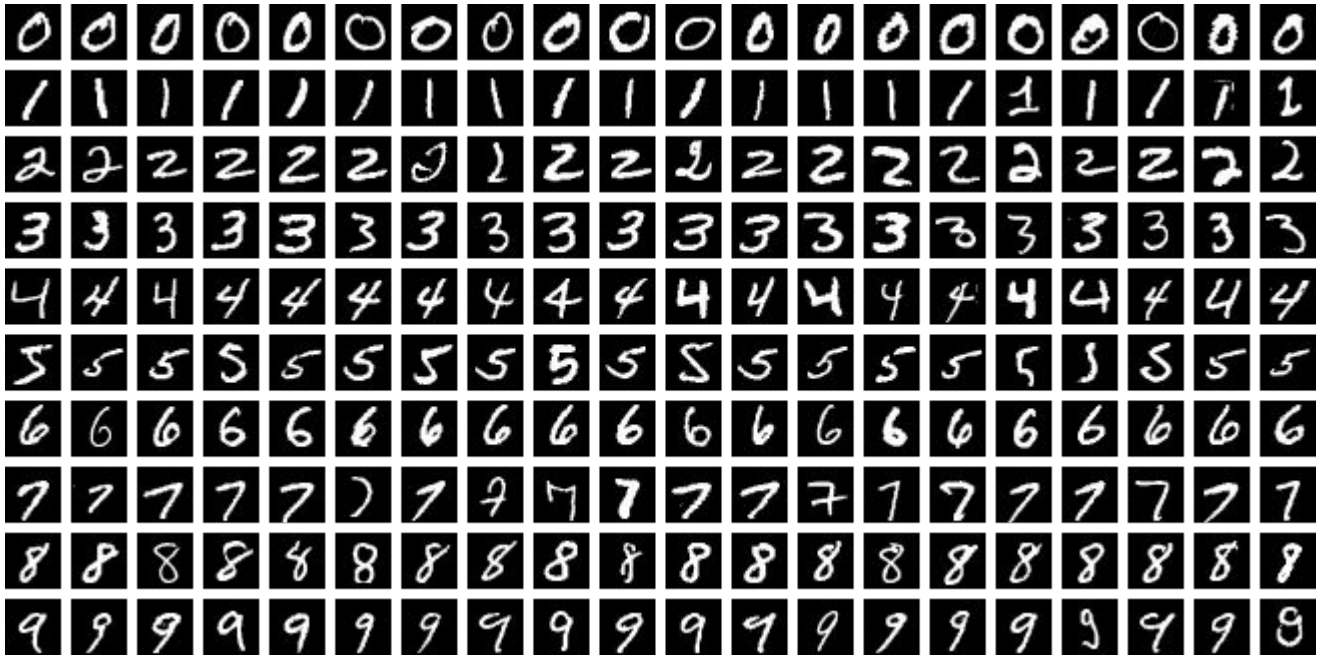


But first let’s see how we can train and generate samples with an unconditional flow model!

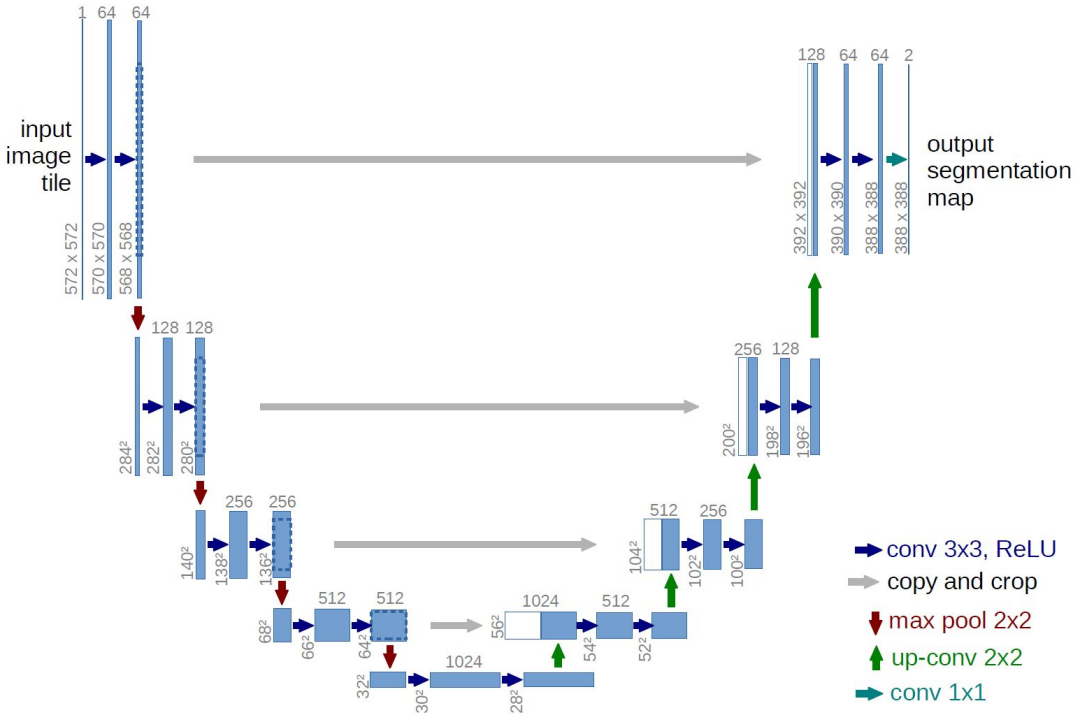
“A picture of a tabby cat”



With PyTorch on the MNIST dataset!



Using a classical U-Net as the parametrization!



Let’s begin with imports

```
# %% 01. Imports
from pathlib import Path  # Dataset and output paths.

import torch  # Tensors, gradients, and optimization.
import torch.nn as nn  # Layers and model classes.
import torch.nn.functional as F  # Resize feature maps in the decoder.
from torch.utils.data import DataLoader  # Shuffle and batch real images.
from torchvision import datasets
from torchvision.transforms import v2  # Convert and normalize image pixels.
from torchvision.utils import save_image  # Export tensors as PNG grids.
```

Let’s begin with imports

- torch provides tensors and gradients
- nn defines layers
- F provides operations such as resizing.

```
# %% 01. Imports
from pathlib import Path  # Dataset and output paths.

import torch  # Tensors, gradients, and optimization.
import torch.nn as nn  # Layers and model classes.
import torch.nn.functional as F  # Resize feature maps in the decoder.
from torch.utils.data import DataLoader  # Shuffle and batch real images.
from torchvision import datasets
from torchvision.transforms import v2  # Convert and normalize image pixels.
from torchvision.utils import save_image  # Export tensors as PNG grids.
```

Let’s begin with imports

- Import batching, image datasets, preprocessing, and image export capabilities from torchvision

```
# %% 01. Imports
from pathlib import Path  # Dataset and output paths.

import torch  # Tensors, gradients, and optimization.
import torch.nn as nn  # Layers and model classes.
import torch.nn.functional as F  # Resize feature maps in the decoder.
from torch.utils.data import DataLoader  # Shuffle and batch real images.
from torchvision import datasets
from torchvision.transforms import v2  # Convert and normalize image pixels.
from torchvision.utils import save_image  # Export tensors as PNG grids.
```

Now set the configuration of the model

Set the batch size, training duration, learning rate, U-Net width, and number of sampling steps.

```
# %% 02. Configuration
BATCH_SIZE = 128  # Images per training batch.
EPOCHS = 20  # Passes through the training set.
LEARNING_RATE = 2e-4  # Optimizer step size.
BASE_CHANNELS = 32  # U-Net widths: 32, 64, 128.
SAMPLE_STEPS = 100  # Euler steps from t=0 to t=1.
NUM_SAMPLES = 16  # Generate a 4-by-4 image grid.
SEED = 7  # Seed for initialization and training.
DATA_DIR = Path("data")  # MNIST download location.
OUTPUT_DIR = Path("flow_matching_outputs")  # Checkpoint and generated image folder.
```

```
DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()  # NVIDIA GPU.
    else "mps" if torch.backends.mps.is_available()  # Apple GPU.
    else "cpu"  # CPU fallback.
)
```

Now set the configuration of the model

Choose your available NVIDIA GPU, Apple GPU, or CPU.

```
# %% 02. Configuration
BATCH_SIZE = 128  # Images per training batch.
EPOCHS = 20  # Passes through the training set.
LEARNING_RATE = 2e-4  # Optimizer step size.
BASE_CHANNELS = 32  # U-Net widths: 32, 64, 128.
SAMPLE_STEPS = 100  # Euler steps from t=0 to t=1.
NUM_SAMPLES = 16  # Generate a 4-by-4 image grid.
SEED = 7  # Seed for initialization and training.
DATA_DIR = Path("data")  # MNIST download location.
OUTPUT_DIR = Path("flow_matching_outputs")  # Checkpoint and generated image folder.
```

```
DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()  # NVIDIA GPU.
    else "mps" if torch.backends.mps.is_available()  # Apple GPU.
    else "cpu"  # CPU fallback.
)
```

Let’s load some images!

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Let’s load some images!

Apply the following transformations in order:

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Let’s load some images!

Apply the following transformations in order:

1. Convert each image into a floating-point tensor with pixel values between zero and one.

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Let’s load some images!

Apply the following transformations in order:

1. Convert each image into a floating-point tensor with pixel values between zero and one.
2. Subtract 0.5 and divide by 0.5, mapping pixel values to -1 through 1.

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Let’s load some images!

Download the MNIST training images and apply these transformations when retrieving each image.

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Let’s load some images!

Create shuffled batches of 128 images, with loading in the main process and pinned memory enabled for CUDA.

```python
# %% 03. Load real images
def make_loader():
    transform = v2.Compose([
        v2.ToImage(),  # Channel-first image: [1, 28, 28].
        v2.ToDtype(torch.float32, scale=True),  # Integer pixels 0-255 -> floats 0-1.
        v2.Normalize(mean=(0.5,), std=(0.5,)),  # Real data pixels: [0, 1] -> [-1, 1].
    ])
    dataset = datasets.MNIST(
        root=DATA_DIR, train=True, download=True, transform=transform,  # Use the training split.
    )
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=True,  # Batch shape: [B, 1, 28, 28].
        num_workers=0, pin_memory=(DEVICE.type == "cuda"),  # Load in the main process.
    )
```

Now let’s sample the conditional path!

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Sample independent standard Gaussian noise with the same shape and device as the real images.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Draw one uniform time per image; shape[0] gives the batch size.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior: same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Add three dimensions so each image’s time broadcasts across its channels, height, and width.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B]
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Weight the noise X_0 by 1 – t and the real image X_1 by t, producing the intermediate image X_t.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Subtract noise from data to obtain the conditional velocity U_t.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

Now let’s sample the conditional path!

Return the intermediate images, their times, and their target velocities.

```python
# %% 04. Sample the conditional path
def sample_conditional_path(x1):  # x1 contains normalized real images.
    x0 = torch.randn_like(x1)  # Independent Gaussian prior; same shape as x1.
    t = torch.rand(x1.shape[0], device=x1.device)  # One random time per image; [B].
    t_image = t[:, None, None, None]  # [B, 1, 1, 1]; share time across pixels.
    xt = (1.0 - t_image) * x0 + t_image * x1  # Noise at t=0; real image at t=1.
    ut = x1 - x0  # Derivative of the straight interpolation.
    return xt, t, ut  # Network input, time, and target velocity.
```

We’re using a U-Net so define the Conv block.

```
# %% 05. Define a simple convolution block
def conv_block(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1), nn.SiLU(),  # Keep H and W.
        nn.Conv2d(out_channels, out_channels, 3, padding=1), nn.SiLU(),  # Refine features.
    )
```

We’re using a U-Net so define the Conv block.

Build a reusable block whose layers run sequentially.

```python
# %% 05. Define a simple convolution block
def conv_block(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1), nn.SiLU(),  # Keep H and W.
        nn.Conv2d(out_channels, out_channels, 3, padding=1), nn.SiLU(),  # Refine features.
    )
```

We’re using a U-Net so define the Conv block.

Build a reusable block whose layers run sequentially.

Apply a 3 x 3 convolution followed by a nonlinear activation; padding preserves the spatial dimensions.

```python
# %% 05. Define a simple convolution block
def conv_block(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1), nn.SiLU(),  # Keep H and W.
        nn.Conv2d(out_channels, out_channels, 3, padding=1), nn.SiLU(),  # Refine features.
    )
```

We’re using a U-Net so define the Conv block.

Build a reusable block whose layers run sequentially.

Apply a 3 x 3 convolution followed by a nonlinear activation; padding preserves the spatial dimensions.

Refine the features with a second convolution and activation.

```
# %% 05. Define a simple convolution block
def conv_block(in_channels, out_channels):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 3, padding=1), nn.SiLU(),  # Keep H and W.
        nn.Conv2d(out_channels, out_channels, 3, padding=1), nn.SiLU(),  # Refine features.
    )
```

And now the full U-Net!

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Define a PyTorch model and initialize its layer and parameter tracking.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Store encoder layers that convert the image and time channels into 32, then 64, feature maps.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Process the lowest-resolution representation using 128 channels.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Combine upsampled and encoder features: 192 channels become 64, then 96 become 32.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Use a 1 x 1 convolution to predict one signed velocity per pixel.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Reduce each 2 x 2 region to its maximum, halving image height and width.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Define the computation PyTorch runs when we call model(xt, t).

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Expand each time value into a constant image channel matching xt.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Join image and time along the channel dimension, producing [B, 2, 28, 28].

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Compute and retain the full-resolution encoder features.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Pool to 14 x 14, compute the second encoder features, and retain them.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Pool to 7 x 7 and process the compressed representation.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Resize to the encoder's height and width, concatenate its saved features, and apply the decoder.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

And now the full U-Net!

Repeat at full resolution using the first encoder skip.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

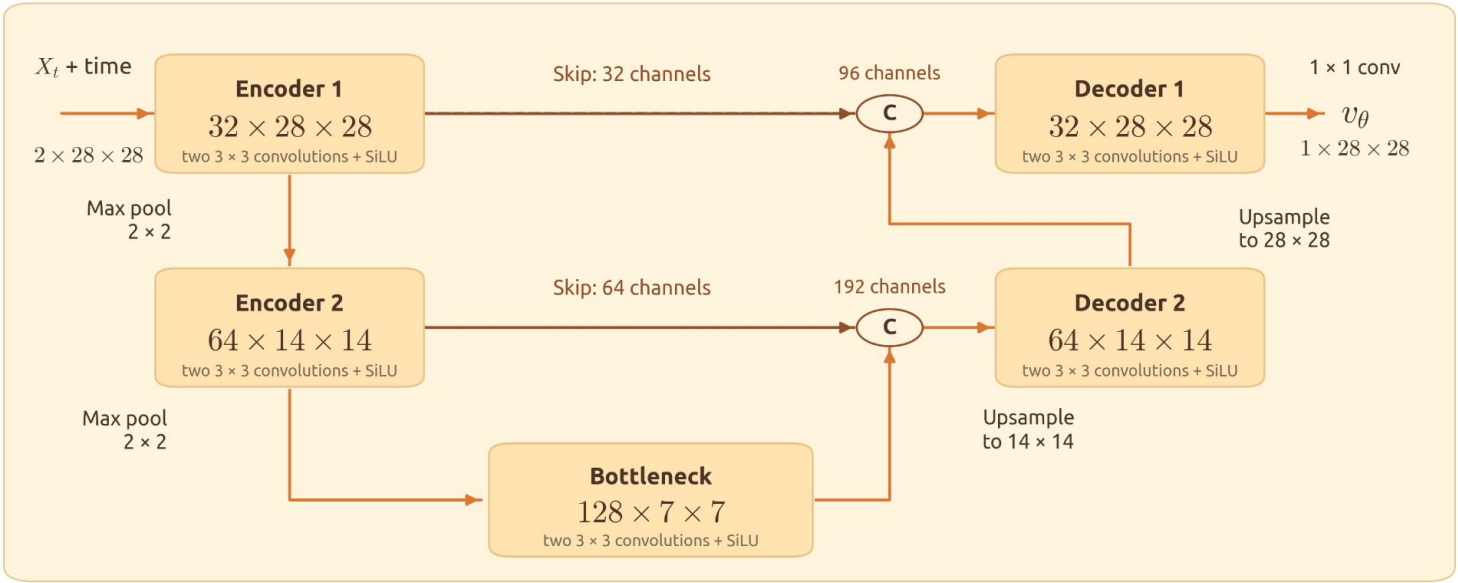
And now the full U-Net!

Return the predicted velocity $v_\theta(X_t, t)$, with the same image shape as the input.

```python
# %% 06. Define the small U-Net
class FlowUNet(nn.Module):
    def __init__(self, base_channels=BASE_CHANNELS):
        super().__init__()
        c = base_channels  # Base width; c=32 by default.
        self.encoder1 = conv_block(2, c)  # Image + time -> c channels.
        self.encoder2 = conv_block(c, 2 * c)  # Encoder: c -> 2c.
        self.middle = conv_block(2 * c, 4 * c)  # Bottleneck: 2c -> 4c.
        self.decoder2 = conv_block(6 * c, 2 * c)  # Upsampled 4c + skip 2c -> 2c.
        self.decoder1 = conv_block(3 * c, c)  # Upsampled 2c + skip c -> c.
        self.output = nn.Conv2d(c, 1, 1)  # One signed velocity per pixel.
        self.pool = nn.MaxPool2d(2)  # Halve the image height and width.

    # %% 07. Compute the velocity field
    def forward(self, xt, t):
        t_image = t[:, None, None, None].expand_as(xt)  # Constant time channel.
        x = torch.cat([xt, t_image], dim=1)  # [B, 2, 28, 28].
        skip1 = self.encoder1(x)  # Keep full-resolution features.
        skip2 = self.encoder2(self.pool(skip1))  # 28x28 -> 14x14.
        x = self.middle(self.pool(skip2))  # 14x14 -> 7x7.
        x = F.interpolate(x, size=skip2.shape[-2:], mode="nearest")  # 7x7 -> 14x14.
        x = self.decoder2(torch.cat([x, skip2], dim=1))  # Restore encoder detail.
        x = F.interpolate(x, size=skip1.shape[-2:], mode="nearest")  # 14x14 -> 28x28.
        x = self.decoder1(torch.cat([x, skip1], dim=1))  # Restore fine detail.
        v_pred = self.output(x)  # Predict v_theta(xt, t).
        return v_pred  # [B, 1, 28, 28], matching xt.
```

So overall, the U-Net downsamples the image into coarse features, upsamples while combining saved encoder features through skip connections, and outputs one signed velocity per pixel.



We have to write the training loop.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Select training mode and initialize the counters used to report average loss.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Retrieve a batch, ignore its digit labels, and move the images to the selected hardware.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Construct intermediate images and exact target velocities using fresh noise and times.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Run the U-Net's forward pass to predict each image's velocity.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Compute the squared velocity error separately at every pixel.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Flatten each image, sum its pixel errors, and average across the batch.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Clear gradients from the previous batch; PyTorch otherwise accumulates them.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Compute the loss gradient with respect to every trainable parameter.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Update the parameters using those gradients.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

We have to write the training loop.

Convert losses to ordinary numbers and report an average weighted by the actual batch sizes.

```python
# %% 08. Train for one epoch
def train_one_epoch(model, loader, optimizer):
    model.train()  # Select training mode.
    total_loss, num_images = 0.0, 0  # Accumulate an image-weighted epoch loss.
    for x1, _ in loader:  # Ignore digit labels: unconditional generation.
        x1 = x1.to(DEVICE)  # Move real images to the model device.
        xt, t, ut = sample_conditional_path(x1)  # Fresh noise and times for this batch.
        v_pred = model(xt, t)  # The model receives only xt and t.
        squared_error = (v_pred - ut).square()  # Velocity error at every pixel.
        loss = squared_error.flatten(1).sum(1).mean()  # Sum pixels, average images.
        optimizer.zero_grad(set_to_none=True)  # Clear old gradients.
        loss.backward()  # Compute gradients with respect to weights.
        optimizer.step()  # Update the U-Net weights.
        total_loss += loss.item() * x1.shape[0]  # Weight by the actual batch size.
        num_images += x1.shape[0]  # Include the smaller final batch.
    return total_loss / num_images  # Average training loss per image.
```

Now time to generate images!

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Disable gradient recording throughout sampling, reducing memory use.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Select evaluation mode and identify the hardware holding the model's parameters.

```
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Initialize one Gaussian noise image for each sample we want to generate.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Divide the interval from zero to one into equal time increments.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Keep an independent CPU copy of the starting noise for visualization.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Advance through the time grid, assigning the current time to every generated image.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Predict the current velocity and add velocity times the time increment: one Euler step.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Predict the current velocity and add velocity times the time increment: one Euler step.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Predict the current velocity and add velocity times the time increment: one Euler step.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Save intermediate states roughly every quarter of the run, including the final state.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Now time to generate images!

Return the generated images and combine their snapshots along a new time dimension.

```python
# Save snapshots every quarter of the default 100-step run; keep ODE states unclipped.
# %% 09. Generate images with Euler integration
@torch.no_grad()  # Sampling needs no gradient tracking.
def sample_images(model, num_samples=NUM_SAMPLES, steps=SAMPLE_STEPS):
    if num_samples < 1 or steps < 1:
        raise ValueError("num_samples and steps must be positive")
    model.eval()  # Select evaluation mode.
    device = next(model.parameters()).device  # Use the model device.
    xt = torch.randn(num_samples, 1, 28, 28, device=device)  # Initial x0.
    dt = 1.0 / steps  # Positive time increment.
    snapshots = [xt.cpu().clone()]  # Keep the initial noise for plotting.
    for step in range(steps):  # Integrate forward in time.
        t = torch.full((num_samples,), step * dt, device=device)  # Shared time grid.
        v_pred = model(xt, t)  # Reevaluate velocity at the current state.
        xt = xt + dt * v_pred  # Euler: state += time increment * velocity.
        if (step + 1) % max(steps // 4, 1) == 0 or step + 1 == steps:
            snapshots.append(xt.cpu().clone())  # Save intermediate and final states.
    return xt, torch.stack(snapshots, dim=1)  # Raw images; [B, snapshots, 1, 28, 28].
```

Finally, set up the entire run!

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Seed random sampling, create the output directory, and prepare the training data.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Instantiate the U-Net and move its parameters to the selected hardware.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Create the optimizer and give it the model parameters to update.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Count all trainable weights and biases; numel() counts entries in each parameter tensor.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Repeat training over the dataset and retain the average loss from each epoch.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Save the learned parameter tensors, loss history, and settings needed to reconstruct the model.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Choose reproducible starting noise and generate images with the trained model.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Save the 16 generated images in a grid with four images per row.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

Finally, set up the entire run!

Save four trajectories, placing successive times across each row.

```python
# %% 10. Set up the complete run
def main():
    torch.manual_seed(SEED)  # Seed training randomness.
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)  # Create the output folder.
    loader = make_loader()  # Download, normalize, and batch MNIST.
    model = FlowUNet().to(DEVICE)  # Create the velocity network.
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=0.0,  # Optimize the flow loss.
    )
    num_parameters = sum(p.numel() for p in model.parameters())  # Count weights and biases.
    print(f"device={DEVICE} parameters={num_parameters:,}")

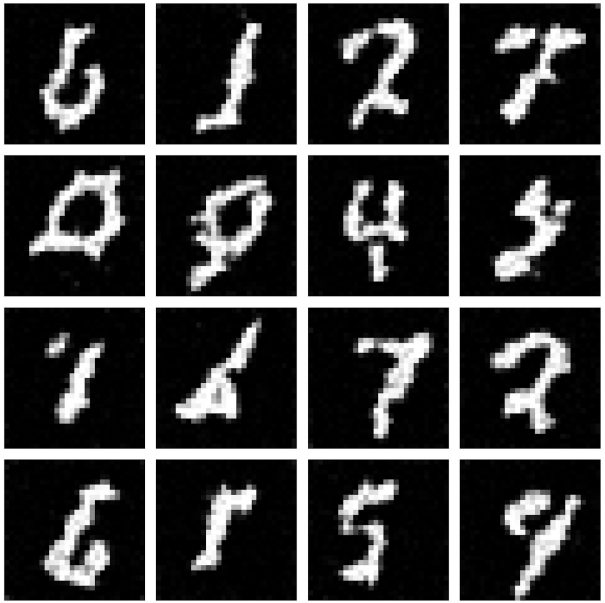
    # %% 11. Train and save the learned parameters
    losses = []  # Record the training loss after each epoch.
    for epoch in range(1, EPOCHS + 1):  # Complete EPOCHS passes through the data.
        loss = train_one_epoch(model, loader, optimizer)  # Run the training loop.
        losses.append(loss)  # Keep the loss history.
        print(f"epoch={epoch:02d} squared_norm_loss={loss:.3f}")
    torch.save({
        "model": model.state_dict(), "losses": losses,  # Weights and training curve.
        "base_channels": BASE_CHANNELS,  # Width needed to reconstruct the model.
        "seed": SEED,  # Record the training seed.
    }, OUTPUT_DIR / "flow_unet_mnist.pt")

    # %% 12. Sample and save images
    torch.manual_seed(SEED + 1)  # Fix the starting noise for generation.
    samples, trajectory = sample_images(model)  # Integrate the learned ODE.
    display_samples = ((samples.cpu() + 1.0) / 2.0).clamp(0.0, 1.0)  # Clip for display only.
    save_image(display_samples, OUTPUT_DIR / "samples.png", nrow=4)  # Final images.
    display_path = ((trajectory[:4] + 1.0) / 2.0).clamp(0.0, 1.0)  # First four sample paths.
    save_image(
        display_path.flatten(0, 1), OUTPUT_DIR / "trajectory.png",  # One row per sample.
        nrow=trajectory.shape[1],  # Time increases across columns.
    )
```

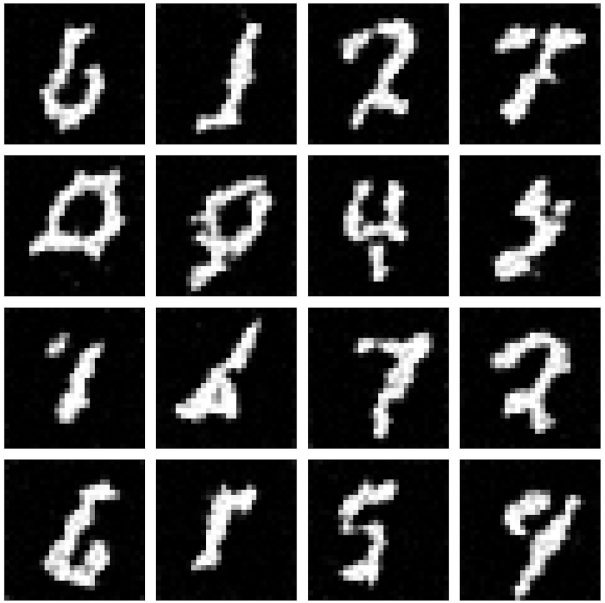
And run the script!

```
# %% 13. Run the script
if __name__ == "__main__":  # Execute when launched directly.
    main()  # Load data, train, sample, and save.
```

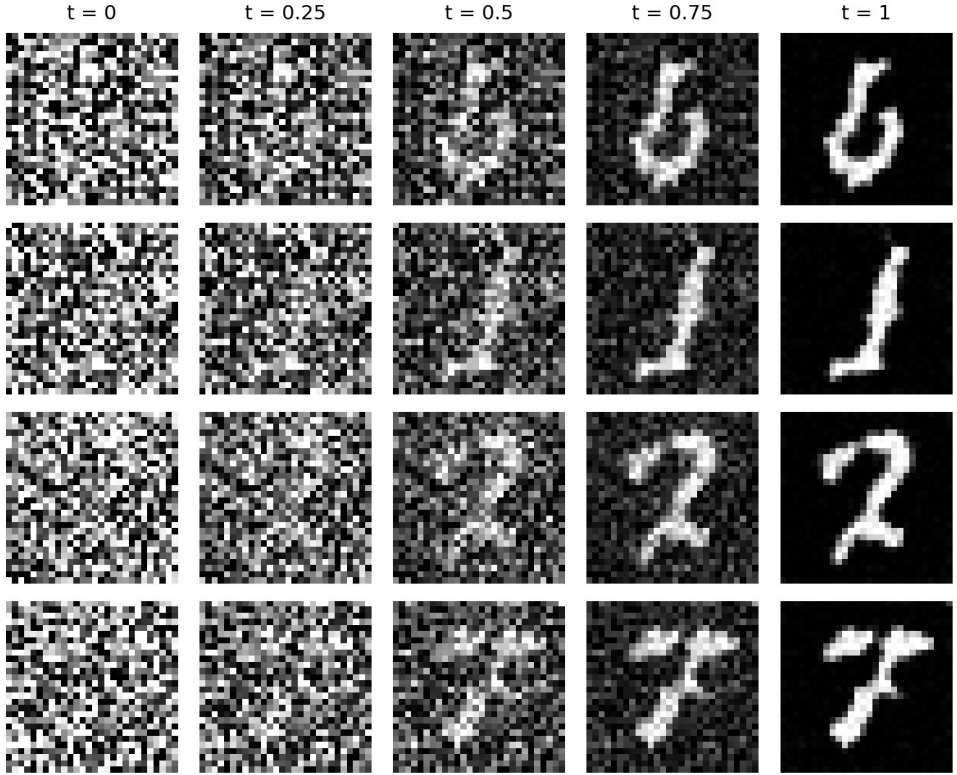
I trained this model with the exact configurations in the script, trained over 5 epochs on my CPU!



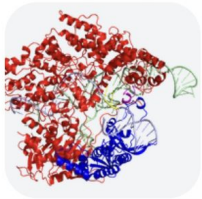
Not too bad!



We can trace the trajectories of the first four selected images at times 0, 0.25, 0.5, 0.75, and 1:



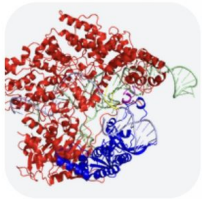
Btw, all lecture code will be available on my lab's HuggingFace!



Programmable Biology Group

ChatterjeeLab

Btw, all lecture code will be available on my lab's HuggingFace!



Programmable Biology Group

`ChatterjeeLab`

Hugging Face | Search models, datasets, users... | Models

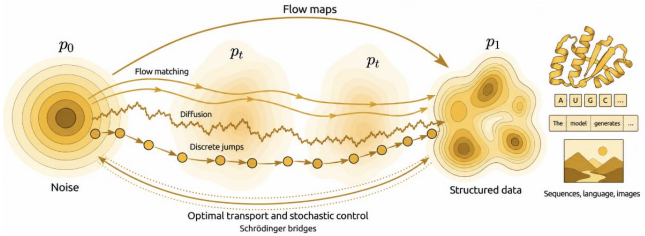
ChatterjeeLab / **CIS6270** | Like 0 | Following Programmable Biol... 139

PyTorch | English | education | flow-matching | diffusion | esm2 | protein-generation | guidance | mn

Model card | Files and versions xet | Community | Settings

Edit model card

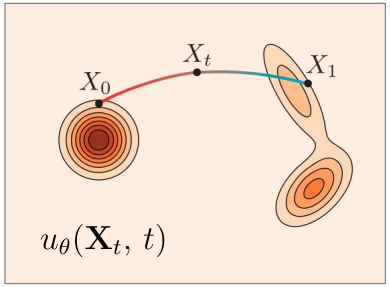
CIS 6270 Course Code

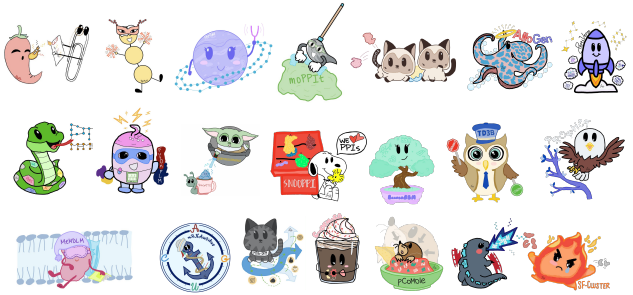


This repository contains code examples for CIS 6270, organized by lecture. We will develop the implementations alongside the course material, which connect the mathematical definitions, training objectives, and sampling equations to the executable PyTorch code herein. Each lecture directory contains its examples, data, and implementation notes.

In summary!

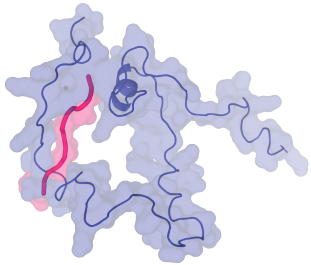
- You're learning a velocity field in pixel space
- That transports samples from noise → data
- Using linear interpolants and flow matching loss
- With a simple, differentiable PyTorch model!

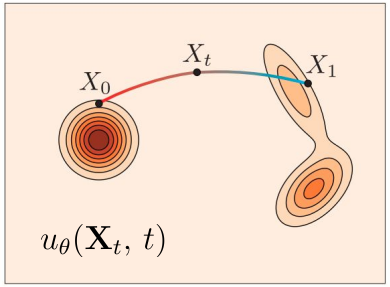


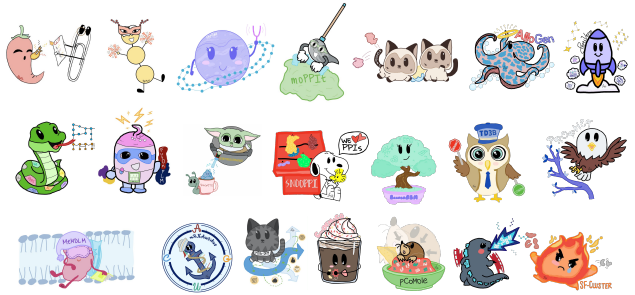


Next, we move to more *stochastic* forms of generative motion!

Penn Engineering
University of Pennsylvania







CIS 6270

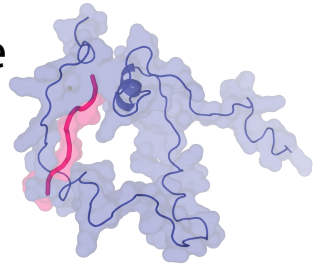
Module 3: Score Matching

Lecture 2.3/3.1: SDEs, Fokker-Planck, and Score Matching

Tue, Sep 15, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



First, recall how flow matching specifies the motion

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

The interpolated state I_t equals one minus time t times the noise sample X_0, plus t times the data sample X_1, with the endpoint pair drawn from the coupling ρ.

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

At an observed state, average the velocities of all endpoint pairs that reach it.

$$b_t(x) = \mathbb{E}[X_1 - X_0 \mid I_t = x]$$

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

At an observed state, average the velocities of all endpoint pairs that reach it.

$$b_t(x) = \mathbb{E}[X_1 - X_0 \mid I_t = x]$$

The marginal velocity $b_t(x)$ equals the average endpoint difference X_1 minus X_0, conditional on the interpolated state I_t being at x.

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

At an observed state, average the velocities of all endpoint pairs that reach it.

$$b_t(x) = \mathbb{E}[X_1 - X_0 \mid I_t = x]$$

The marginal velocity $b_t(x)$ equals the average endpoint difference X_1 minus X_0, conditional on the interpolated state I_t being at x.

We get a simple velocity target from each pair.

First, recall how flow matching specifies the motion

Pair a noise sample X_0 with a data sample X_1. The coupling ρ specifies how we draw the pair.

$$I_t = (1-t)X_0 + tX_1, \qquad (X_0, X_1) \sim \rho$$

At an observed state, average the velocities of all endpoint pairs that reach it.

$$b_t(x) = \mathbb{E}[X_1 - X_0 \mid I_t = x]$$

The marginal velocity $b_t(x)$ equals the average endpoint difference X_1 minus X_0, conditional on the interpolated state I_t being at x.

We get a simple velocity target from each pair. Then, we can use standard squared-error training to learn the average velocity at the current state.

After training, the learned velocity determines the trajectory

After training, the learned velocity determines the trajectory

Sample an initial noise state and solve the learned ODE.

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \qquad X_0 \sim p_{\text{noise}}$$

After training, the learned velocity determines the trajectory

Sample an initial noise state and solve the learned ODE.

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \qquad X_0 \sim p_{\text{noise}}$$

The state's rate of change dX_t/dt equals the learned velocity v_θ at the current state X_t and time t, with the initial state X_0 sampled from the noise distribution p_{noise}.

After training, the learned velocity determines the trajectory

Sample an initial noise state and solve the learned ODE.

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \qquad X_0 \sim p_{\text{noise}}$$

The state's rate of change dX_t/dt equals the learned velocity v_θ at the current state X_t and time t, with the initial state X_0 sampled from the noise distribution p_{noise}.

Once the initial state is fixed, repeating the ODE calculation follows the same path.

After training, the learned velocity determines the trajectory

Sample an initial noise state and solve the learned ODE.

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \qquad X_0 \sim p_{\text{noise}}$$

The state's rate of change dX_t/dt equals the learned velocity v_θ at the current state X_t and time t, with the initial state X_0 sampled from the noise distribution p_{noise}.

Once the initial state is fixed, repeating the ODE calculation follows the same path. All sampling randomness entered at the start.

We can also build the path by gradually adding noise

We can also build the path by gradually adding noise

Diffusion begins with clean data and repeatedly adds controlled Gaussian perturbations.

We can also build the path by gradually adding noise

Diffusion begins with clean data and repeatedly adds controlled Gaussian perturbations. With a suitable drift and noise schedule, the distribution approaches a simple Gaussian prior.

We can also build the path by gradually adding noise

Diffusion begins with clean data and repeatedly adds controlled Gaussian perturbations. With a suitable drift and noise schedule, the distribution approaches a simple Gaussian prior.

We choose these perturbations ourselves, so we can generate noisy training examples and know exactly which noise we added.

Learning to reverse that noising process gives us a generator

Learning to reverse that noising process gives us a generator

A noisy state can come from many different clean samples.

Learning to reverse that noising process gives us a generator

A noisy state can come from many different clean samples. To choose a reverse direction, we need to know how the noisy-data density changes near the current state.

Learning to reverse that noising process gives us a generator

A noisy state can come from many different clean samples. To choose a reverse direction, we need to know how the noisy-data density changes near the current state.

Its log-density gradient is the score.

Learning to reverse that noising process gives us a generator

A noisy state can come from many different clean samples. To choose a reverse direction, we need to know how the noisy-data density changes near the current state.

Its log-density gradient is the score. We will learn that score from the known perturbations and use it to move from the Gaussian prior back toward data.

An SDE describes the small random steps that create this path

An SDE describes the small random steps that create this path

Each step combines a predictable displacement with a fresh random perturbation.

An SDE describes the small random steps that create this path

Each step combines a predictable displacement with a fresh random perturbation. A stochastic differential equation describes the limit as the steps become small.

An SDE describes the small random steps that create this path

Each step combines a predictable displacement with a fresh random perturbation. A stochastic differential equation describes the limit as the steps become small.

First we will connect those random trajectories to their evolving density.

An SDE describes the small random steps that create this path

Each step combines a predictable displacement with a fresh random perturbation. A stochastic differential equation describes the limit as the steps become small.

First we will connect those random trajectories to their evolving density. Then we can derive the reverse process, recover an ODE with the same densities, and carry guidance into both samplers.

Describe the motion with drift and Brownian increments

Describe the motion with drift and Brownian increments

Let X_t be a state with d coordinates. The drift f specifies its average motion, and the scalar g sets the strength of standard Brownian noise W_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

Describe the motion with drift and Brownian increments

Let X_t be a state with d coordinates. The drift f specifies its average motion, and the scalar g sets the strength of standard Brownian noise W_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

The change in state dX_t equals the drift $f(X_t, t)$ times the time increment dt, plus the noise scale g(t) times the Brownian increment dW_t.

Describe the motion with drift and Brownian increments

Let X_t be a state with d coordinates. The drift f specifies its average motion, and the scalar g sets the strength of standard Brownian noise W_t.

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t$$

The change in state dX_t equals the drift $f(X_t, t)$ times the time increment dt, plus the noise scale g(t) times the Brownian increment dW_t.

The drift sets the average displacement. The random increment creates a spread of possible next states.

A Brownian increment grows with the square root of time

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

The Brownian increment ΔW equals the square root of the interval length h times a standard Gaussian draw ε, whose covariance is the identity matrix I.

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Multiplying a random vector by √h multiplies its covariance by h.

$$\mathrm{Cov}(\Delta W) = hI$$

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Multiplying a random vector by √h multiplies its covariance by h.

$$\mathrm{Cov}(\Delta W) = hI$$

The covariance of the Brownian increment ΔW equals the elapsed time h times the identity matrix I.

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Multiplying a random vector by √h multiplies its covariance by h.

$$\mathrm{Cov}(\Delta W) = hI$$

The covariance of the Brownian increment ΔW equals the elapsed time h times the identity matrix I.

Halving the interval halves the variance.

A Brownian increment grows with the square root of time

Brownian motion starts at zero and has independent increments. Over an interval of length h, draw a standard Gaussian vector ε.

$$\Delta W = \sqrt{h}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Multiplying a random vector by √h multiplies its covariance by h.

$$\mathrm{Cov}(\Delta W) = hI$$

The covariance of the Brownian increment ΔW equals the elapsed time h times the identity matrix I.

Halving the interval halves the variance. The typical displacement decreases by a factor of the square root of two.

Approximate the SDE with a finite drift-and-noise step

Approximate the SDE with a finite drift-and-noise step

Euler-Maruyama takes a finite step of length h from state x_k at time t_k. Draw independent standard Gaussian noise ε_k at each step.

$$x_{k+1} = x_k + f(x_k, t_k)h + g(t_k)\sqrt{h}\,\varepsilon_k$$

Approximate the SDE with a finite drift-and-noise step

Euler-Maruyama takes a finite step of length h from state x_k at time t_k. Draw independent standard Gaussian noise ε_k at each step.

$$x_{k+1} = x_k + f(x_k, t_k)h + g(t_k)\sqrt{h}\,\varepsilon_k$$

The next state x_{k+1} equals the current state x_k, plus the drift $f(x_k, t_k)$ times the step size h, plus the noise scale $g(t_k)$ times the square root of h and the Gaussian draw ε_k.

Approximate the SDE with a finite drift-and-noise step

Euler-Maruyama takes a finite step of length h from state x_k at time t_k. Draw independent standard Gaussian noise ε_k at each step.

$$x_{k+1} = x_k + f(x_k, t_k)h + g(t_k)\sqrt{h}\,\varepsilon_k$$

The next state x_{k+1} equals the current state x_k, plus the drift $f(x_k, t_k)$ times the step size h, plus the noise scale $g(t_k)$ times the square root of h and the Gaussian draw ε_k.

We can reuse the same initial state and obtain different trajectories because every step draws new noise.

Let’s calculate that step with actual numbers

Let’s calculate that step with actual numbers

Start at $x_k = 2$, with drift $f = -1$ and noise scale $g = 0.5$. Take a time step $h = 0.04$ and use the standard Gaussian noise draw $\varepsilon_k = 1.5$.

$$x_{k+1} = 2 + (-1)(0.04) + (0.5)(0.2)(1.5) = 2.11$$

Let’s calculate that step with actual numbers

Start at $x_k = 2$, with drift $f = -1$ and noise scale $g = 0.5$. Take a time step $h = 0.04$ and use the standard Gaussian noise draw $\varepsilon_k = 1.5$.

$$x_{k+1} = 2 + (-1)(0.04) + (0.5)(0.2)(1.5) = 2.11$$

The next state x_{k+1} equals two minus 0.04 from the drift, plus 0.15 from the sampled noise, giving 2.11.

Let’s calculate that step with actual numbers

Start at $x_k = 2$, with drift $f = -1$ and noise scale $g = 0.5$. Take a time step $h = 0.04$ and use the standard Gaussian noise draw $\varepsilon_k = 1.5$.

$$x_{k+1} = 2 + (-1)(0.04) + (0.5)(0.2)(1.5) = 2.11$$

The next state x_{k+1} equals two minus 0.04 from the drift, plus 0.15 from the sampled noise, giving 2.11.

The drift alone ends at 1.96.

Let’s calculate that step with actual numbers

Start at $x_k = 2$, with drift $f = -1$ and noise scale $g = 0.5$. Take a time step $h = 0.04$ and use the standard Gaussian noise draw $\varepsilon_k = 1.5$.

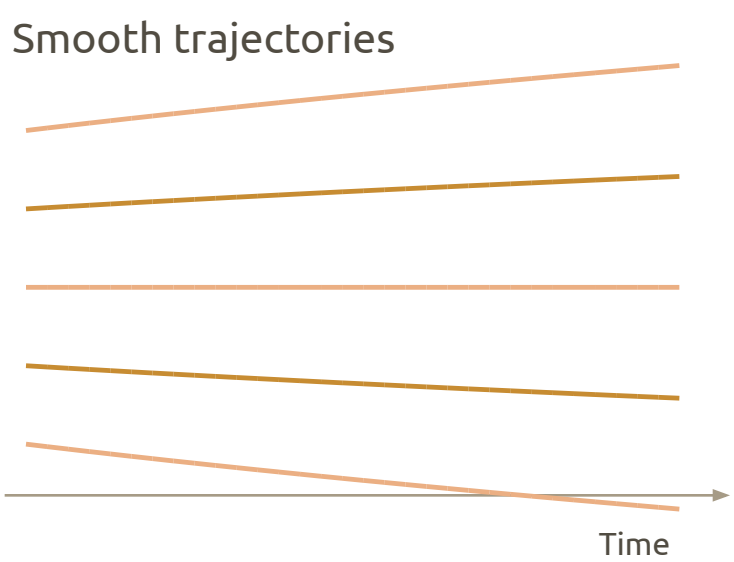
$$x_{k+1} = 2 + (-1)(0.04) + (0.5)(0.2)(1.5) = 2.11$$

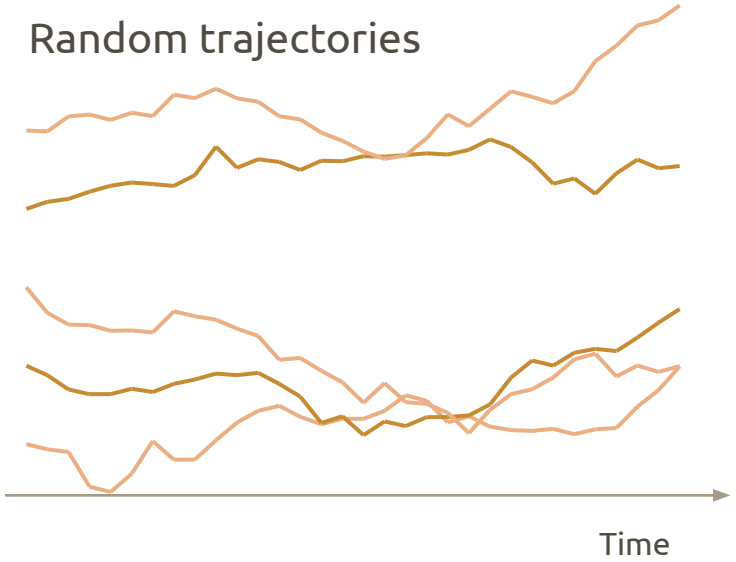
The next state x_{k+1} equals two minus 0.04 from the drift, plus 0.15 from the sampled noise, giving 2.11.

The drift alone ends at 1.96. This particular noise draw raises the state to 2.11, so individual paths can move against the average direction.

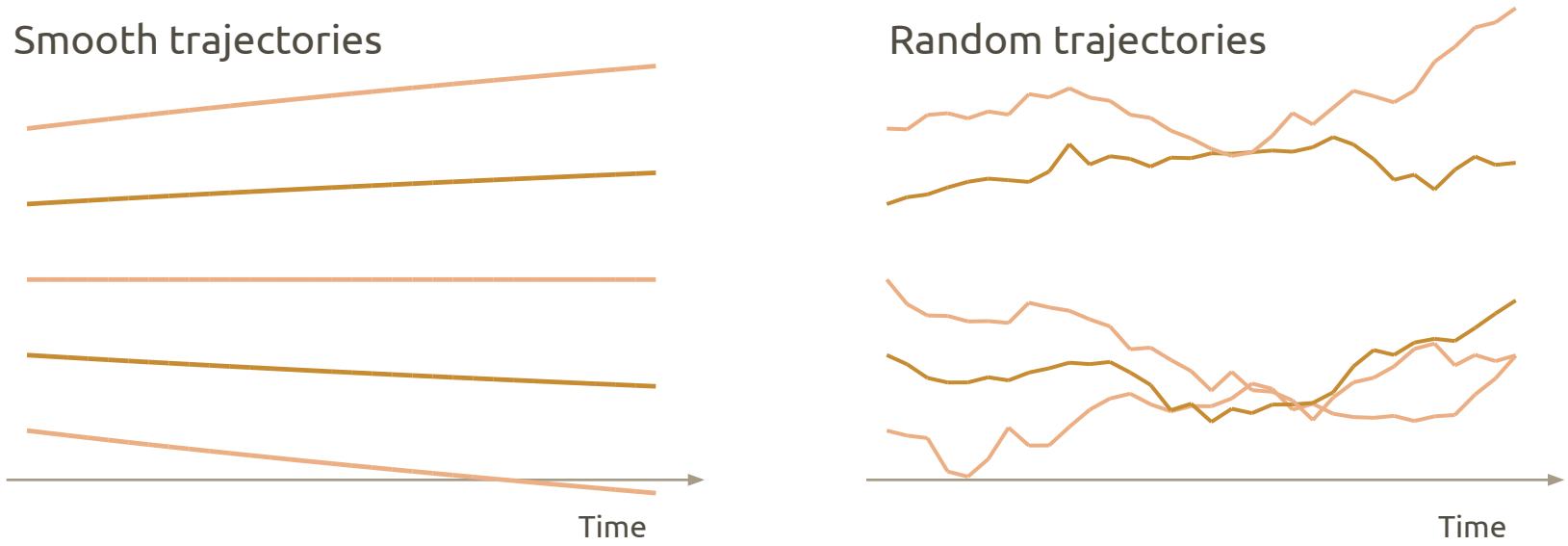
Fresh random increments change each trajectory

Fresh random increments change each trajectory



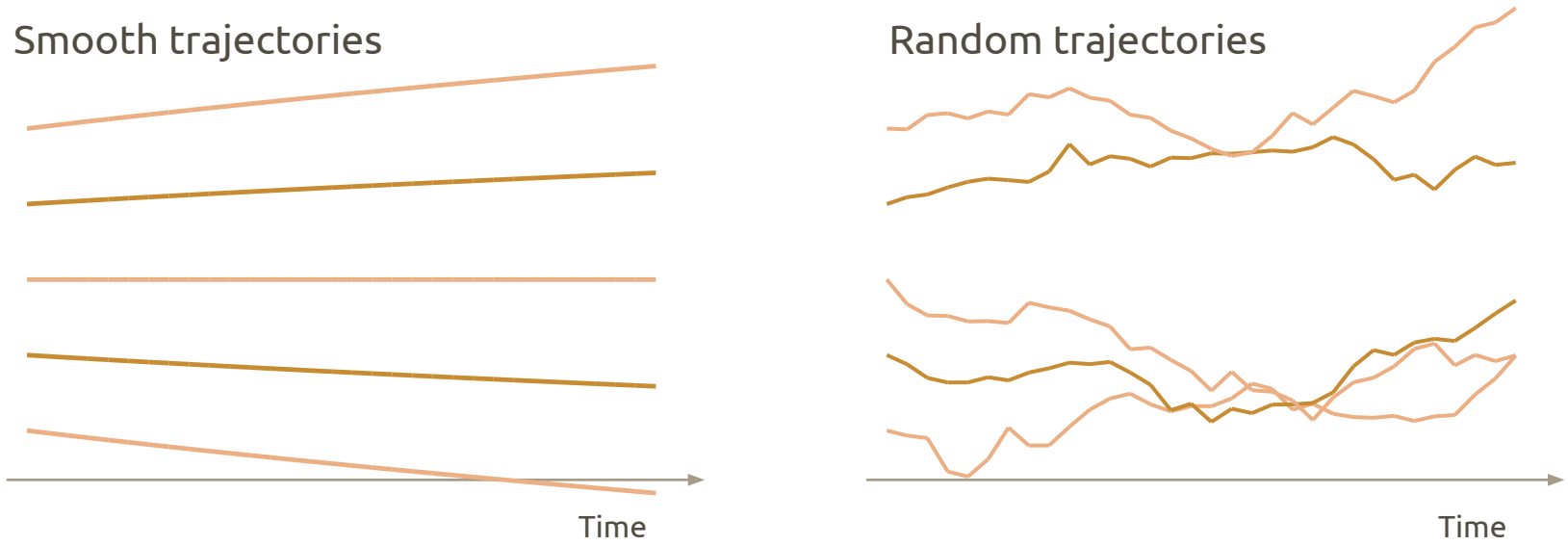


Fresh random increments change each trajectory



The smooth paths follow their velocities from fixed starting states. Each random path receives fresh perturbations along the way.

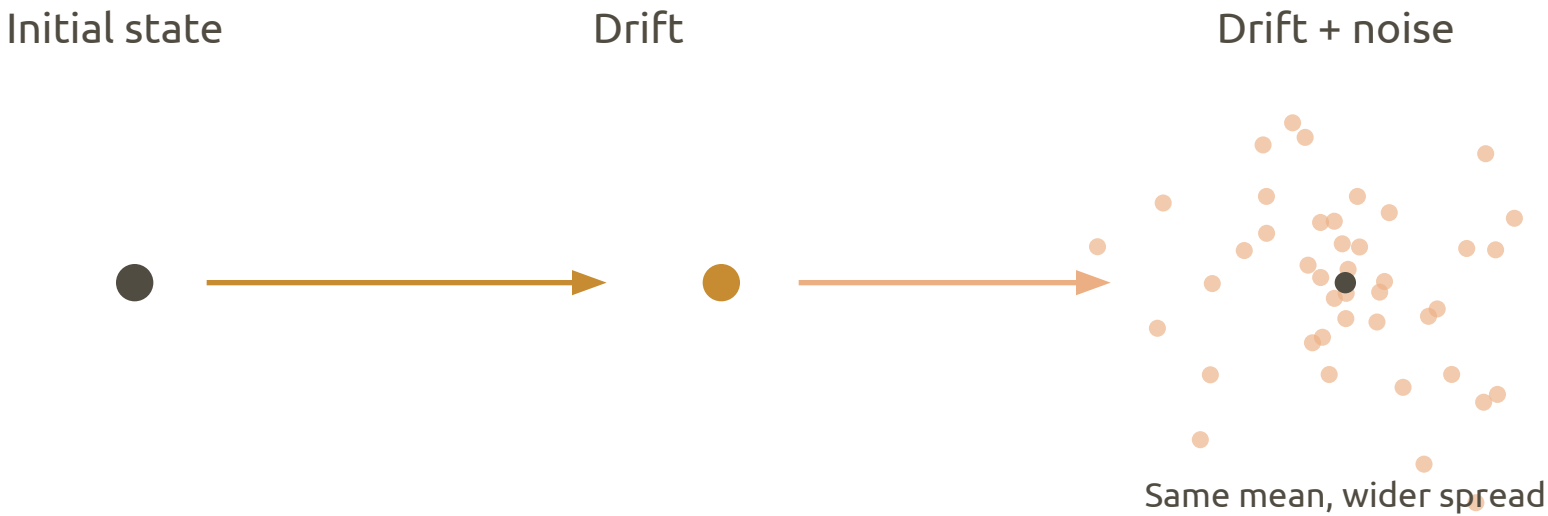
Fresh random increments change each trajectory



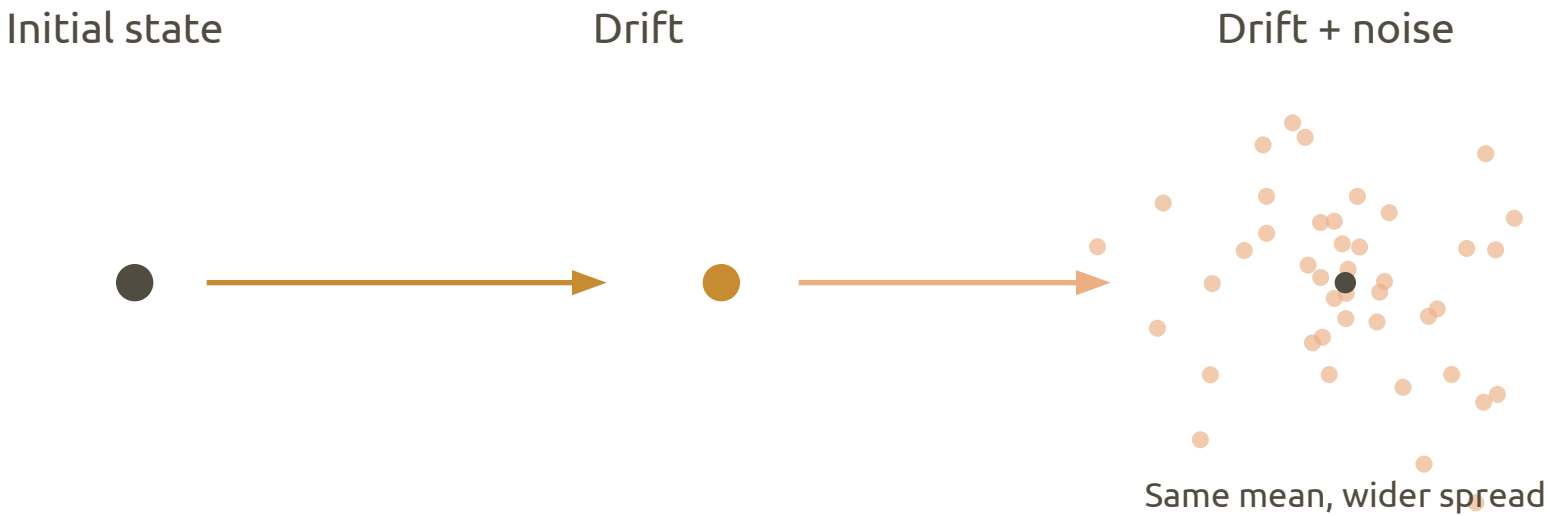
The smooth paths follow their velocities from fixed starting states. Each random path receives fresh perturbations along the way. We now need to track how the whole population spreads.

The drift moves the center while noise widens the cloud

The drift moves the center while noise widens the cloud



The drift moves the center while noise widens the cloud



Start every particle at the same point. The drift shifts their average position, while the independent noise draws spread their positions apart.

A restoring drift prevents unlimited spreading

A restoring drift prevents unlimited spreading

Use a restoring rate κ > 0 and a constant noise scale σ.

$$dX_t = -\kappa X_t\,dt + \sigma\,dW_t$$

A restoring drift prevents unlimited spreading

Use a restoring rate $\kappa > 0$ and a constant noise scale σ.

$$dX_t = -\kappa X_t \, dt + \sigma \, dW_t$$

The state increment dX_t equals negative restoring rate κ times the current state X_t and dt, plus the noise scale σ times the Brownian increment dW_t.

A restoring drift prevents unlimited spreading

Use a restoring rate $\kappa > 0$ and a constant noise scale σ.

$$dX_t = -\kappa X_t\,dt + \sigma\,dW_t$$

Starting from x_0, the Ornstein-Uhlenbeck process has a Gaussian transition.

$$X_t \mid x_0 \sim \mathcal{N}\left(e^{-\kappa t}x_0, \frac{\sigma^2}{2\kappa}(1 - e^{-2\kappa t})I\right)$$

A restoring drift prevents unlimited spreading

Use a restoring rate κ > 0 and a constant noise scale σ.

$$dX_t = -\kappa X_t\,dt + \sigma\,dW_t$$

Starting from x_0, the Ornstein-Uhlenbeck process has a Gaussian transition.

$$X_t \mid x_0 \sim \mathcal{N}\left(e^{-\kappa t}x_0, \frac{\sigma^2}{2\kappa}(1 - e^{-2\kappa t})I\right)$$

The state X_t conditional on its start x_0 is Gaussian, with mean e to negative κt times x_0 and covariance σ^2 divided by twice κ, times one minus e to negative 2κt, times I.

A restoring drift prevents unlimited spreading

Use a restoring rate κ > 0 and a constant noise scale σ.

$$dX_t = -\kappa X_t\,dt + \sigma\,dW_t$$

Starting from x_0, the Ornstein-Uhlenbeck process has a Gaussian transition.

$$X_t \mid x_0 \sim \mathcal{N}\left(e^{-\kappa t}x_0, \frac{\sigma^2}{2\kappa}(1 - e^{-2\kappa t})I\right)$$

The state X_t conditional on its start x_0 is Gaussian, with mean e to negative κt times x_0 and covariance σ^2 divided by twice κ, times one minus e to negative 2κt, times I.

The contribution from the initial state decays while noise accumulates.

A restoring drift prevents unlimited spreading

Use a restoring rate κ > 0 and a constant noise scale σ.

$$dX_t = -\kappa X_t \, dt + \sigma \, dW_t$$

Starting from x_0, the Ornstein-Uhlenbeck process has a Gaussian transition.

$$X_t \mid x_0 \sim \mathcal{N}\left(e^{-\kappa t} x_0, \frac{\sigma^2}{2\kappa}(1 - e^{-2\kappa t}) I \right)$$

The state X_t conditional on its start x_0 is Gaussian, with mean e to negative κt times x_0 and covariance σ^2 divided by twice κ, times one minus e to negative 2κt, times I.

The contribution from the initial state decays while noise accumulates. We can choose the drift and noise scales so the variance approaches one.

Let’s calculate the remaining signal and accumulated variance

Let’s calculate the remaining signal and accumulated variance

Start at $x_0 = 2$, with restoring rate $\kappa = 1$ and noise scale $\sigma = \sqrt{2}$. Evaluate the mean and variance at time $t = \log 2$.

$$\mathbb{E}[X_t \mid x_0] = \tfrac{1}{2}(2) = 1,$$

$$\mathrm{Var}(X_t \mid x_0) = 1 - \tfrac{1}{4} = 0.75$$

Let’s calculate the remaining signal and accumulated variance

Start at $x_0 = 2$, with restoring rate $\kappa = 1$ and noise scale $\sigma = \sqrt{2}$. Evaluate the mean and variance at time $t = \log 2$.

$$\mathbb{E}[X_t \mid x_0] = \tfrac{1}{2}(2) = 1,$$

$$\mathrm{Var}(X_t \mid x_0) = 1 - \tfrac{1}{4} = 0.75$$

The conditional mean $E[X_t \mid x_0]$ equals one-half times two, giving one, and the conditional variance $\mathrm{Var}(X_t \mid x_0)$ equals one minus one-quarter, giving 0.75.

Let’s calculate the remaining signal and accumulated variance

Start at $x_0 = 2$, with restoring rate $\kappa = 1$ and noise scale $\sigma = \sqrt{2}$. Evaluate the mean and variance at time $t = \log 2$.

$$\mathbb{E}[X_t \mid x_0] = \tfrac{1}{2}(2) = 1,$$

$$\mathrm{Var}(X_t \mid x_0) = 1 - \tfrac{1}{4} = 0.75$$

The conditional mean $E[X_t \mid x_0]$ equals one-half times two, giving one, and the conditional variance $\mathrm{Var}(X_t \mid x_0)$ equals one minus one-quarter, giving 0.75.

Half of the initial signal remains.

Let’s calculate the remaining signal and accumulated variance

Start at $x_0 = 2$, with restoring rate $\kappa = 1$ and noise scale $\sigma = \sqrt{2}$. Evaluate the mean and variance at time $t = \log 2$.

$$\mathbb{E}[X_t \mid x_0] = \tfrac{1}{2}(2) = 1,$$

$$\mathrm{Var}(X_t \mid x_0) = 1 - \tfrac{1}{4} = 0.75$$

The conditional mean $E[X_t \mid x_0]$ equals one-half times two, giving one, and the conditional variance $\mathrm{Var}(X_t \mid x_0)$ equals one minus one-quarter, giving 0.75.

Half of the initial signal remains. As we continue, the mean approaches zero and the variance approaches one.

Now we need to track a function of a random state

Now we need to track a function of a random state

Start with one random state at time t. Choose a smooth probe φ that returns one number from the current state.

$$X_t \longmapsto \phi(X_t)$$

Now we need to track a function of a random state

Start with one random state at time t. Choose a smooth probe φ that returns one number from the current state.

$$X_t \longmapsto \phi(X_t)$$

The random state enters the smooth function φ, which can measure position, squared position, or any other smooth quantity.

Now we need to track a function of a random state

Now average that measurement over many particles whose positions follow the density at time t.

$$X_t \sim p_t$$

$$\mathbb{E}[\phi(X_t)] = \int \phi(x) p_t(x)\, dx$$

Now we need to track a function of a random state

Now average that measurement over many particles whose positions follow the density at time t.

$$X_t \sim p_t$$

$$\mathbb{E}[\phi(X_t)] = \int \phi(x) p_t(x)\, dx$$

The expected probe value equals the integral of φ(x) weighted by the density at time t.

Now we need to track a function of a random state

Now average that measurement over many particles whose positions follow the density at time t.

$$X_t \sim p_t$$

$$\mathbb{E}[\phi(X_t)] = \int \phi(x) p_t(x)\, dx$$

Itô's lemma follows the probe along one random path. Averaging its change reveals how the entire density evolves.

Squared Brownian increments accumulate over time

Squared Brownian increments accumulate over time

Over interval k of width h, Brownian motion adds a Gaussian increment with variance h.

$$\Delta W_k \sim \mathcal{N}(0, h), \qquad h = \frac{T}{N}$$

Squared Brownian increments accumulate over time

Over interval k of width h, Brownian motion adds a Gaussian increment with variance h.

$$\Delta W_k \sim \mathcal{N}(0, h), \qquad h = \frac{T}{N}$$

The Brownian increment ΔW in interval k has mean zero and variance h, where h equals total time T divided by N intervals.

Squared Brownian increments accumulate over time

A variance of h gives a standard deviation, or typical movement size, equal to the square root of h.

$$\Delta W_k \sim \mathcal{N}(0, h)$$

$$\Delta W_k = \sqrt{h}\,\varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, 1)$$

Squared Brownian increments accumulate over time

A variance of h gives a standard deviation, or typical movement size, equal to the square root of h.

$$\Delta W_k \sim \mathcal{N}(0, h)$$

$$\Delta W_k = \sqrt{h}\,\varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, 1)$$

Write the Brownian increment as square root of h times ε in interval k, where ε is a standard Gaussian random variable.

Squared Brownian increments accumulate over time

Now square the random increment to see which time scale survives.

$$\Delta W_k = \sqrt{h}\,\varepsilon_k$$

$$(\Delta W_k)^2 = h\,\varepsilon_k^2$$

$$\varepsilon_k^2 = \mathcal{O}(1) \quad \Longrightarrow \quad (\Delta W_k)^2 = \mathcal{O}(h)$$

Squared Brownian increments accumulate over time

Now square the random increment to see which time scale survives.

$$\Delta W_k = \sqrt{h}\,\varepsilon_k$$

$$(\Delta W_k)^2 = h\,\varepsilon_k^2$$

$$\varepsilon_k^2 = \mathcal{O}(1) \quad \Longrightarrow \quad (\Delta W_k)^2 = \mathcal{O}(h)$$

The squared increment equals h times ε squared. Because ε squared stays of order one, the squared Brownian increment stays of order h.

Squared Brownian increments accumulate over time

Now square the random increment to see which time scale survives.

$$\Delta W_k = \sqrt{h}\,\varepsilon_k$$

$$(\Delta W_k)^2 = h\,\varepsilon_k^2$$

$$\varepsilon_k^2 = \mathcal{O}(1) \quad \Longrightarrow \quad (\Delta W_k)^2 = \mathcal{O}(h)$$

The squared increment equals h times ε squared. Because ε squared stays of order one, the squared Brownian increment stays of order h.

For h = 0.01, the typical displacement is 0.1. Squaring 0.1 returns 0.01, so the curvature term remains on the same scale as time.

Squared Brownian increments accumulate over time

Divide elapsed time T into N intervals of length h = T/N. Write ΔW_k for the one-dimensional Brownian increment in interval k.

$$\sum_{k=0}^{N-1} (\Delta W_k)^2 \longrightarrow T \quad \text{as } N \to \infty$$

Squared Brownian increments accumulate over time

Divide elapsed time T into N intervals of length h = T/N. Write ΔW_k for the one-dimensional Brownian increment in interval k.

$$\sum_{k=0}^{N-1} (\Delta W_k)^2 \longrightarrow T \quad \text{as } N \to \infty$$

The sum of squared Brownian increments, $(\Delta W_k)^2$, from interval zero through N minus one approaches the elapsed time T as the number of intervals N increases without bound.

Squared Brownian increments accumulate over time

Divide elapsed time T into N intervals of length h = T/N. Write ΔW_k for the one-dimensional Brownian increment in interval k.

$$\sum_{k=0}^{N-1} (\Delta W_k)^2 \longrightarrow T \quad \text{as } N \to \infty$$

The sum of squared Brownian increments, $(\Delta W_k)^2$, from interval zero through N minus one approaches the elapsed time T as the number of intervals N increases without bound.

A typical squared increment has size h, and there are T/h increments.

Squared Brownian increments accumulate over time

Divide elapsed time T into N intervals of length h = T/N. Write ΔW_k for the one-dimensional Brownian increment in interval k.

$$\sum_{k=0}^{N-1} (\Delta W_k)^2 \longrightarrow T \quad \text{as } N \to \infty$$

The sum of squared Brownian increments, $(\Delta W_k)^2$, from interval zero through N minus one approaches the elapsed time T as the number of intervals N increases without bound.

A typical squared increment has size h, and there are T/h increments. Their total stays of order T as the intervals shrink.

That surviving square changes the chain rule

That surviving square changes the chain rule

Use φ(x) = x squared to see why Taylor expansion needs a curvature correction.

$$\phi(x) = x^2$$

$$\phi(X + \Delta X) - \phi(X) = 2X\Delta X + (\Delta X)^2$$

That surviving square changes the chain rule

Use φ(x) = x squared to see why Taylor expansion needs a curvature correction.

$$\phi(x) = x^2$$

$$\phi(X + \Delta X) - \phi(X) = 2X\Delta X + (\Delta X)^2$$

For the square function, changing X by ΔX changes φ by 2XΔX plus ΔX squared. The first term comes from slope, while the second records curvature.

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

The function change Δφ is approximately the slope ϕ_x times the state increment ΔX, plus one-half the curvature ϕ_{xx} times the squared increment $(\Delta X)^2$.

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Insert the SDE increment and write the Brownian part using its square-root time scale.

$$\Delta X = f\,\Delta t + g\,\Delta W = f\,\Delta t + g\sqrt{\Delta t}\,\varepsilon$$

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Insert the SDE increment and write the Brownian part using its square-root time scale.

$$\Delta X = f\,\Delta t + g\,\Delta W = f\,\Delta t + g\sqrt{\Delta t}\,\varepsilon$$

The state increment ΔX equals drift f times Δt plus noise scale g times square root of Δt times ε.

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Square the full state increment and group each term by its power of Δt.

$$\Delta X = f\,\Delta t + g\sqrt{\Delta t}\,\varepsilon$$

$$(\Delta X)^2 = f^2(\Delta t)^2 + 2fg(\Delta t)^{3/2}\varepsilon + g^2\Delta t\,\varepsilon^2$$

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Square the full state increment and group each term by its power of Δt.

$$\Delta X = f\,\Delta t + g\sqrt{\Delta t}\,\varepsilon$$

$$(\Delta X)^2 = f^2(\Delta t)^2 + 2fg(\Delta t)^{3/2}\varepsilon + g^2\Delta t\,\varepsilon^2$$

Squaring produces terms of order Δt squared, Δt to the three-halves, and Δt. Only the final noise-squared term remains at first order in time.

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Let Δt shrink and keep only terms of order Δt.

$$(dX)^2 = g(t)^2\, dt$$

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Let Δt shrink and keep only terms of order Δt.

$$(dX)^2 = g(t)^2\, dt$$

The higher-order terms vanish, while ε squared remains of order one. Therefore the squared state increment contributes g squared times Δt.

That surviving square changes the chain rule

For a smooth function φ, write ϕ_x for its slope and ϕ_{xx} for its curvature. Expand around the current state.

$$\Delta\phi \approx \phi_x \Delta X + \tfrac{1}{2}\phi_{xx}(\Delta X)^2$$

Let Δt shrink and keep only terms of order Δt.

$$(dX)^2 = g(t)^2\, dt$$

The higher-order terms vanish, while ε squared remains of order one. Therefore the squared state increment contributes g squared times Δt.

Brownian motion makes the second-order Taylor term as large as the ordinary first-order drift term. This surviving curvature correction becomes the extra term in Itô’s lemma.

Itô’s lemma keeps that curvature contribution

Itô’s lemma keeps that curvature contribution

For a smooth function φ(x, t), write φ_t for its time derivative, φ_x for its slope, and φ_{xx} for its curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right)dt + g\phi_x\, dW_t$$

Itô’s lemma keeps that curvature contribution

For a smooth function $\varphi(x, t)$, write φ_t for its time derivative, φ_x for its slope, and φ_{xx} for its curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right) dt + g\phi_x\, dW_t$$

The change in the function $d\varphi$ equals its time derivative φ_t plus the drift f times its slope φ_x plus one-half the squared noise scale g^2 times its curvature φ_{xx}, all multiplied by dt, plus $g\varphi_x$ times the Brownian increment dW_t.

Itô’s lemma keeps that curvature contribution

For a smooth function φ(x, t), write φ_t for its time derivative, φ_x for its slope, and φ_{xx} for its curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right)dt + g\phi_x\, dW_t$$

The change in the function dφ equals its time derivative φ_t plus the drift f times its slope φ_x plus one-half the squared noise scale g^2 times its curvature φ_{xx}, all multiplied by dt, plus $g\varphi_x$ times the Brownian increment dW_t.

Curvature changes the average function value even when the noise has zero mean.

Itô’s lemma keeps that curvature contribution

For a smooth function φ(x, t), write φ_t for its time derivative, φ_x for its slope, and φ_{xx} for its curvature.

$$d\phi = \left(\phi_t + f\phi_x + \tfrac{1}{2}g^2\phi_{xx}\right) dt + g\phi_x\, dW_t$$

The change in the function dφ equals its time derivative φ_t plus the drift f times its slope φ_x plus one-half the squared noise scale g^2 times its curvature φ_{xx}, all multiplied by dt, plus $g\varphi_x$ times the Brownian increment dW_t.

Curvature changes the average function value even when the noise has zero mean. When we average over trajectories, this term contributes to diffusion in the density equation.

Squaring Brownian motion makes the correction visible

Squaring Brownian motion makes the correction visible

Set $X_t = W_t$ and $\varphi(x) = x^2$. Then $\varphi_x = 2x$ and $\varphi_{xx} = 2$.

$$d(W_t^2) = 2W_t\,dW_t + dt$$

Squaring Brownian motion makes the correction visible

Set $X_t = W_t$ and $\varphi(x) = x^2$. Then $\varphi_x = 2x$ and $\varphi_{xx} = 2$.

$$d(W_t^2) = 2W_t\,dW_t + dt$$

The change in squared Brownian position $d(W_t^2)$ equals twice the current position W_t times the Brownian increment dW_t, plus the elapsed time increment dt.

Squaring Brownian motion makes the correction visible

Set $X_t = W_t$ and $\varphi(x) = x^2$. Then $\varphi_x = 2x$ and $\varphi_{xx} = 2$.

$$d(W_t^2) = 2W_t\,dW_t + dt$$

The change in squared Brownian position $d(W_t^2)$ equals twice the current position W_t times the Brownian increment dW_t, plus the elapsed time increment dt.

The extra dt term explains why squared distance grows even though the Brownian increment has mean zero.

Taking the expectation recovers the variance we know

Taking the expectation recovers the variance we know

The stochastic integral has zero expectation under the usual integrability conditions.

$$\frac{d}{dt}\mathbb{E}[W_t^2] = 1, \qquad \mathbb{E}[W_t^2] = t$$

Taking the expectation recovers the variance we know

The stochastic integral has zero expectation under the usual integrability conditions.

$$\frac{d}{dt}\mathbb{E}[W_t^2] = 1, \qquad \mathbb{E}[W_t^2] = t$$

The expected squared Brownian position $E[W_t^2]$ increases at a rate of one, so starting from zero gives $E[W_t^2]$ equal to the elapsed time t.

Taking the expectation recovers the variance we know

The stochastic integral has zero expectation under the usual integrability conditions.

$$\frac{d}{dt}\mathbb{E}[W_t^2] = 1, \qquad \mathbb{E}[W_t^2] = t$$

The expected squared Brownian position $E[W_t^2]$ increases at a rate of one, so starting from zero gives $E[W_t^2]$ equal to the elapsed time t.

At t = 0.25, the mean squared position is 0.25 and the standard deviation is 0.5.

Taking the expectation recovers the variance we know

The stochastic integral has zero expectation under the usual integrability conditions.

$$\frac{d}{dt}\mathbb{E}[W_t^2] = 1, \qquad \mathbb{E}[W_t^2] = t$$

The expected squared Brownian position $E[W_t^2]$ increases at a rate of one, so starting from zero gives $E[W_t^2]$ equal to the elapsed time t.

At t = 0.25, the mean squared position is 0.25 and the standard deviation is 0.5. Omitting the curvature term would lose this growth.

Average Itô’s formula to track a smooth function of the state

Average Itô's formula to track a smooth function of the state

In d dimensions, collect the first derivatives in the gradient $\nabla\varphi$ and add the second derivatives in the Laplacian $\Delta\varphi$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

Average Itô's formula to track a smooth function of the state

In d dimensions, collect the first derivatives in the gradient $\nabla\varphi$ and add the second derivatives in the Laplacian $\Delta\varphi$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

The rate of change of the expected measurement $E[\varphi(X_t)]$ equals the expectation of the drift f dotted with the gradient $\nabla\varphi$, plus one-half the squared noise scale g^2 times the Laplacian $\Delta\varphi$.

Average Itô's formula to track a smooth function of the state

In d dimensions, collect the first derivatives in the gradient $\nabla\varphi$ and add the second derivatives in the Laplacian $\Delta\varphi$.

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f \cdot \nabla\phi + \tfrac{1}{2}g^2\Delta\phi\right]$$

The rate of change of the expected measurement $E[\varphi(X_t)]$ equals the expectation of the drift f dotted with the gradient $\nabla\varphi$, plus one-half the squared noise scale g^2 times the Laplacian $\Delta\varphi$.

With this equation, we can calculate how the SDE changes any smooth measurement of the population.

Integration by parts transfers the derivatives to density

Integration by parts transfers the derivatives to density

First replace the expectation with an integral against the density.

$$\begin{aligned} &\frac{d}{dt}\int \phi(x) p_t(x)\, dx \\ &= \int \Big[f \cdot \nabla \phi + \tfrac{1}{2} g^2 \Delta \phi \Big] p_t(x)\, dx \end{aligned}$$

Integration by parts transfers the derivatives to density

First replace the expectation with an integral against the density.

$$\frac{d}{dt}\int \phi(x) p_t(x)\,dx = \int \Big[f \cdot \nabla \phi + \tfrac{1}{2} g^2 \Delta \phi \Big] p_t(x)\,dx$$

The time derivative of the average probe value equals the integral of the Itô drift and curvature terms, weighted by the density at time t.

Integration by parts transfers the derivatives to density

Move the derivative in the drift term from φ onto the probability flux.

$$\frac{d}{dt}\int \phi p_t \, dx = \int \left[f \cdot \nabla \phi + \frac{1}{2} g^2 \Delta \phi \right] p_t \, dx$$

$$\int f \cdot \nabla \phi \, p_t \, dx = -\int \phi \, \nabla \cdot (f p_t) \, dx$$

Integration by parts transfers the derivatives to density

Move the derivative in the drift term from φ onto the probability flux.

$$\frac{d}{dt}\int \phi p_t \, dx = \int \left[f \cdot \nabla \phi + \frac{1}{2} g^2 \Delta \phi \right] p_t \, dx$$

$$\int f \cdot \nabla \phi \, p_t \, dx = -\int \phi \, \nabla \cdot (f p_t) \, dx$$

Integration by parts changes the drift integral into negative φ times the divergence of f times the density at time t.

Integration by parts transfers the derivatives to density

Move both derivatives in the Laplacian term from φ onto the density.

$$\int f \cdot \nabla \phi \, p_t \, dx = -\int \phi \, \nabla \cdot (f p_t) \, dx$$

$$\int \Delta \phi \, p_t \, dx = \int \phi \, \Delta p_t \, dx$$

$$\text{boundary terms} = 0$$

Integration by parts transfers the derivatives to density

Move both derivatives in the Laplacian term from φ onto the density.

$$\int f \cdot \nabla \phi \, p_t \, dx = -\int \phi \, \nabla \cdot (f p_t) \, dx$$

$$\int \Delta \phi \, p_t \, dx = \int \phi \, \Delta p_t \, dx$$

$$\text{boundary terms} = 0$$

Two integrations by parts move the Laplacian from φ to the density. The two minus signs cancel, and the boundary terms vanish.

Integration by parts transfers the derivatives to density

Combine the transported drift term and the diffusive curvature term.

$$\int \phi \, \partial_t p_t \, dx = \int \phi \left[-\nabla \cdot (f p_t) + \tfrac{1}{2} g^2 \Delta p_t \right] dx$$

Integration by parts transfers the derivatives to density

Combine the transported drift term and the diffusive curvature term.

$$\int \phi \, \partial_t p_t \, dx = \int \phi \left[-\nabla \cdot (f p_t) + \tfrac{1}{2} g^2 \Delta p_t \right] dx$$

The integral of the measurement φ times the density change $\partial_t p_t$ equals the integral of φ times negative divergence of $f p_t$ plus one-half g^2 times the density Laplacian Δp_t.

Integration by parts transfers the derivatives to density

Combine the transported drift term and the diffusive curvature term.

$$\int \phi \, \partial_t p_t \, dx = \int \phi \left[-\nabla \cdot (f p_t) + \tfrac{1}{2} g^2 \Delta p_t \right] dx$$

The integral of the measurement φ times the density change $\partial_t p_t$ equals the integral of φ times negative divergence of $f p_t$ plus one-half g^2 times the density Laplacian Δp_t.

Choose a smooth test function φ to measure local changes in density.

Integration by parts transfers the derivatives to density

Combine the transported drift term and the diffusive curvature term.

$$\int \phi \, \partial_t p_t \, dx = \int \phi \left[-\nabla \cdot (f p_t) + \tfrac{1}{2} g^2 \Delta p_t \right] dx$$

The integral of the measurement φ times the density change $\partial_t p_t$ equals the integral of φ times negative divergence of $f p_t$ plus one-half g^2 times the density Laplacian Δp_t.

Choose a smooth test function φ to measure local changes in density. Because the integral identity holds for every such choice, we can identify the density's time derivative.

Fokker-Planck describes how the whole density changes

Fokker-Planck describes how the whole density changes

The equality holds for every smooth test function. The density itself therefore satisfies the Fokker-Planck equation.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

Fokker-Planck describes how the whole density changes

The equality holds for every smooth test function. The density itself therefore satisfies the Fokker-Planck equation.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

The density change $\partial_t p_t$ equals negative divergence of the probability flux $f p_t$, plus one-half the squared noise scale $g(t)^2$ times the density Laplacian Δp_t.

Fokker-Planck describes how the whole density changes

The equality holds for every smooth test function. The density itself therefore satisfies the Fokker-Planck equation.

$$\partial_t p_t = -\nabla \cdot (f p_t) + \tfrac{1}{2} g(t)^2 \Delta p_t$$

The density change $\partial_t p_t$ equals negative divergence of the probability flux $f p_t$, plus one-half the squared noise scale $g(t)^2$ times the density Laplacian Δp_t.

A density peak has negative curvature, so diffusion lowers that peak and spreads probability into nearby regions.

Let’s calculate how diffusion lowers a Gaussian peak

Let’s calculate how diffusion lowers a Gaussian peak

Start with a standard Gaussian density, use zero drift, and set the Brownian noise scale to one. Calculate the density and curvature at the peak, x = 0.

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989, \qquad p_0''(0) = -p_0(0) \approx -0.3989$$

Let’s calculate how diffusion lowers a Gaussian peak

Start with a standard Gaussian density, use zero drift, and set the Brownian noise scale to one. Calculate the density and curvature at the peak, x = 0.

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989, \qquad p_0''(0) = -p_0(0) \approx -0.3989$$

At the peak, the density is about 0.3989 and its second derivative is negative 0.3989.

Let’s calculate how diffusion lowers a Gaussian peak

Start with a standard Gaussian density, use zero drift, and set the Brownian noise scale to one. Calculate the density and curvature at the peak, x = 0.

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989, \qquad p_0''(0) = -p_0(0) \approx -0.3989$$

Multiply the curvature by one-half the squared noise scale to obtain the density’s rate of change.

$$\partial_t p_t(0)|_{t=0} = \tfrac{1}{2} p_0''(0) \approx -0.1995$$

Let’s calculate how diffusion lowers a Gaussian peak

Start with a standard Gaussian density, use zero drift, and set the Brownian noise scale to one. Calculate the density and curvature at the peak, x = 0.

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989, \qquad p_0''(0) = -p_0(0) \approx -0.3989$$

Multiply the curvature by one-half the squared noise scale to obtain the density’s rate of change.

$$\partial_t p_t(0)|_{t=0} = \tfrac{1}{2} p_0''(0) \approx -0.1995$$

At time zero, the peak density decreases at a rate of about 0.1995 per unit time.

Let’s calculate how diffusion lowers a Gaussian peak

Start with a standard Gaussian density, use zero drift, and set the Brownian noise scale to one. Calculate the density and curvature at the peak, x = 0.

$$p_0(0) = \frac{1}{\sqrt{2\pi}} \approx 0.3989, \qquad p_0''(0) = -p_0(0) \approx -0.3989$$

Multiply the curvature by one-half the squared noise scale to obtain the density’s rate of change.

$$\partial_t p_t(0)|_{t=0} = \tfrac{1}{2} p_0''(0) \approx -0.1995$$

At time zero, the peak density decreases at a rate of about 0.1995 per unit time.

Over a short interval of 0.1, the first-order estimate drops from 0.3989 to about 0.3790. Probability spreads outward while total probability remains one.

For diffusion, our clock starts at the data

For diffusion, our clock starts at the data

We use t = 0 for clean data and t = T for the noisy prior. The forward process adds noise as t increases.

For diffusion, our clock starts at the data

We use t = 0 for clean data and t = T for the noisy prior. The forward process adds noise as t increases.

Generation will traverse this path from T back to zero.

For diffusion, our clock starts at the data

We use t = 0 for clean data and t = T for the noisy prior. The forward process adds noise as t increases.

Generation will traverse this path from T back to zero. Later, we will use a separate forward-running generation clock to compare with flow matching.

Choose the drift and noise scales for variance-preserving diffusion

Choose the drift and noise scales for variance-preserving diffusion

Choose a positive noising rate β(t). It controls both the restoring drift and the injected noise.

$$dX_t = -\tfrac{1}{2}\beta(t)X_t\,dt + \sqrt{\beta(t)}\,dW_t$$

Choose the drift and noise scales for variance-preserving diffusion

Choose a positive noising rate β(t). It controls both the restoring drift and the injected noise.

$$dX_t = -\tfrac{1}{2}\beta(t)X_t\,dt + \sqrt{\beta(t)}\,dW_t$$

The state increment dX_t equals negative one-half the noising rate β(t) times the current state X_t and dt, plus the square root of β(t) times the Brownian increment dW_t.

Choose the drift and noise scales for variance-preserving diffusion

Choose a positive noising rate β(t). It controls both the restoring drift and the injected noise.

$$dX_t = -\tfrac{1}{2}\beta(t)X_t\,dt + \sqrt{\beta(t)}\,dW_t$$

The state increment dX_t equals negative one-half the noising rate β(t) times the current state X_t and dt, plus the square root of β(t) times the Brownian increment dW_t.

A standard Gaussian stays unchanged under this process.

Choose the drift and noise scales for variance-preserving diffusion

Choose a positive noising rate β(t). It controls both the restoring drift and the injected noise.

$$dX_t = -\tfrac{1}{2}\beta(t)X_t\,dt + \sqrt{\beta(t)}\,dW_t$$

The state increment dX_t equals negative one-half the noising rate β(t) times the current state X_t and dt, plus the square root of β(t) times the Brownian increment dW_t.

A standard Gaussian stays unchanged under this process. Other distributions gradually lose their original structure.

We can sample any noise level directly

We can sample any noise level directly

Accumulate the noising rate β and use it to define the signal coefficient a_t.

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}$$

We can sample any noise level directly

Accumulate the noising rate β and use it to define the signal coefficient a_t.

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}$$

The accumulated noising B(t) equals the integral of β(s) from zero to t, and the signal coefficient a_t equals e to negative one-half B(t).

We can sample any noise level directly

Accumulate the noising rate β and use it to define the signal coefficient a_t.

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}$$

At this time, combine a clean sample with an independent Gaussian draw ε.

$$X_t = a_t X_0 + \sqrt{1 - a_t^2}\, \varepsilon$$

We can sample any noise level directly

Accumulate the noising rate β and use it to define the signal coefficient a_t.

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}$$

At this time, combine a clean sample with an independent Gaussian draw ε.

$$X_t = a_t X_0 + \sqrt{1 - a_t^2}\, \varepsilon$$

The noisy state X_t equals the signal coefficient a_t times the clean state X_0, plus the square root of one minus a_t^2 times the Gaussian noise ε.

We can sample any noise level directly

Accumulate the noising rate β and use it to define the signal coefficient a_t.

$$B(t) = \int_0^t \beta(s)\, ds, \qquad a_t = e^{-B(t)/2}$$

At this time, combine a clean sample with an independent Gaussian draw ε.

$$X_t = a_t X_0 + \sqrt{1 - a_t^2}\, \varepsilon$$

The noisy state X_t equals the signal coefficient a_t times the clean state X_0, plus the square root of one minus a_t^2 times the Gaussian noise ε.

During training, we can sample directly at a chosen noise level. We can skip the earlier noising steps.

With unit-covariance data, this path preserves covariance

With unit-covariance data, this path preserves covariance

For independent clean data and Gaussian noise, the covariance adds.

$$\mathrm{Cov}(X_t) = a_t^2 \,\mathrm{Cov}(X_0) + (1 - a_t^2)I$$

With unit-covariance data, this path preserves covariance

For independent clean data and Gaussian noise, the covariance adds.

$$\mathrm{Cov}(X_t) = a_t^2 \,\mathrm{Cov}(X_0) + (1 - a_t^2)I$$

The noisy-state covariance Cov(X_t) equals the squared signal coefficient a_t^2 times the clean covariance Cov(X_0), plus one minus a_t^2 times the identity matrix I.

With unit-covariance data, this path preserves covariance

For independent clean data and Gaussian noise, the covariance adds.

$$\mathrm{Cov}(X_t) = a_t^2 \,\mathrm{Cov}(X_0) + (1 - a_t^2)I$$

The noisy-state covariance Cov(X_t) equals the squared signal coefficient a_t^2 times the clean covariance Cov(X_0), plus one minus a_t^2 times the identity matrix I.

If the clean data has identity covariance, the noisy state does too.

With unit-covariance data, this path preserves covariance

For independent clean data and Gaussian noise, the covariance adds.

$$\mathrm{Cov}(X_t) = a_t^2 \,\mathrm{Cov}(X_0) + (1 - a_t^2)I$$

The noisy-state covariance Cov(X_t) equals the squared signal coefficient a_t^2 times the clean covariance Cov(X_0), plus one minus a_t^2 times the identity matrix I.

If the clean data has identity covariance, the noisy state does too. For general data, the covariance moves toward the identity as the signal coefficient decays.

Let’s keep one noisy sample through the next calculations

Let’s keep one noisy sample through the next calculations

Start with clean sample $x_0 = 10$. Use signal retention coefficient $a_t = 0.8$ and noise standard deviation $\sigma_t = 0.6$, with a Gaussian noise draw $\varepsilon = -1$.

$$x_t = (0.8)(10) + (0.6)(-1) = 7.4, \qquad q_t(x \mid x_0) = \mathcal{N}(8, 0.36)$$

Let’s keep one noisy sample through the next calculations

Start with clean sample $x_0 = 10$. Use signal retention coefficient $a_t = 0.8$ and noise standard deviation $\sigma_t = 0.6$, with a Gaussian noise draw $\varepsilon = -1$.

$$x_t = (0.8)(10) + (0.6)(-1) = 7.4, \qquad q_t(x \mid x_0) = \mathcal{N}(8, 0.36)$$

The noisy state x_t equals 0.8 times ten plus 0.6 times negative one, giving 7.4, and its corruption density q_t has mean eight and variance 0.36.

Let’s keep one noisy sample through the next calculations

Start with clean sample x_0 = 10. Use signal retention coefficient a_t = 0.8 and noise standard deviation σ_t = 0.6, with a Gaussian noise draw ε = −1.

$$x_t = (0.8)(10) + (0.6)(-1) = 7.4, \qquad q_t(x \mid x_0) = \mathcal{N}(8, 0.36)$$

The noisy state x_t equals 0.8 times ten plus 0.6 times negative one, giving 7.4, and its corruption density q_t has mean eight and variance 0.36.

The draw lies 0.6 below its conditional mean.

Let’s keep one noisy sample through the next calculations

Start with clean sample x_0 = 10. Use signal retention coefficient a_t = 0.8 and noise standard deviation σ_t = 0.6, with a Gaussian noise draw ε = −1.

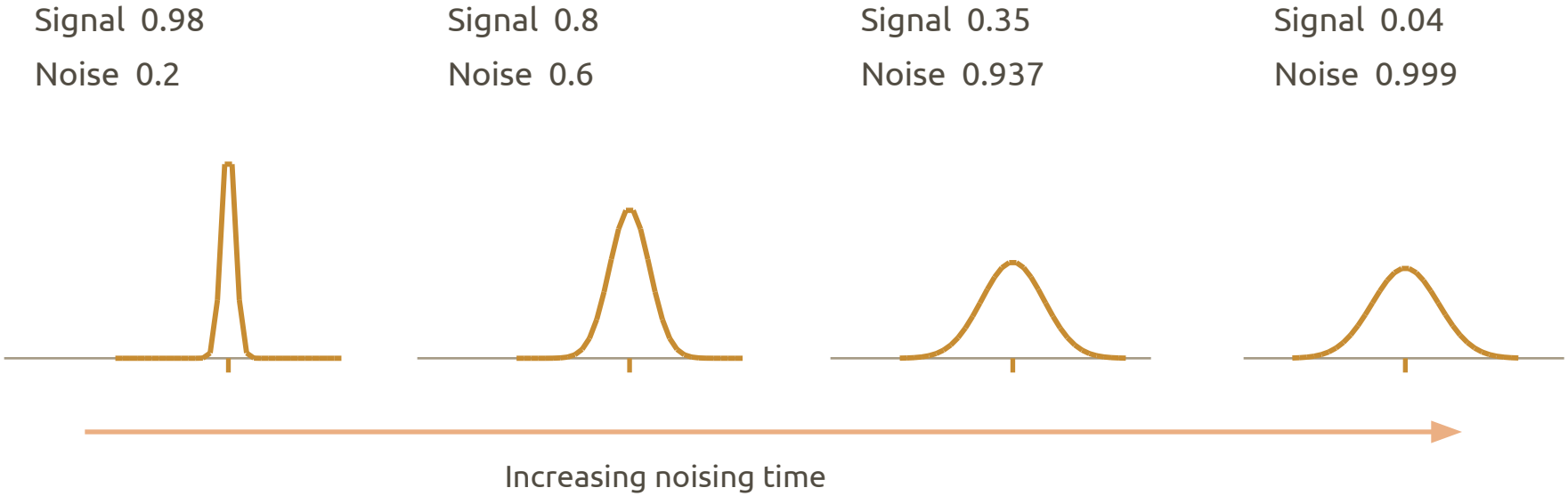
$$x_t = (0.8)(10) + (0.6)(-1) = 7.4, \qquad q_t(x \mid x_0) = \mathcal{N}(8, 0.36)$$

The noisy state x_t equals 0.8 times ten plus 0.6 times negative one, giving 7.4, and its corruption density q_t has mean eight and variance 0.36.

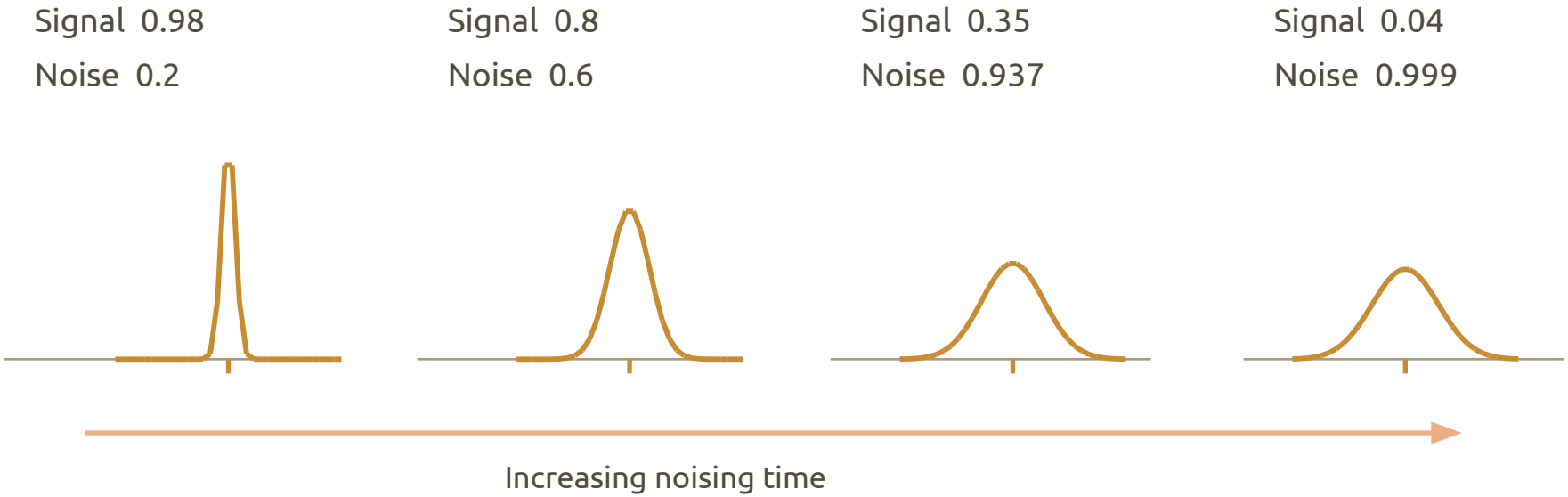
The draw lies 0.6 below its conditional mean. We will use that displacement to calculate the denoising target.

As signal fades, the conditional cloud expands

As signal fades, the conditional cloud expands

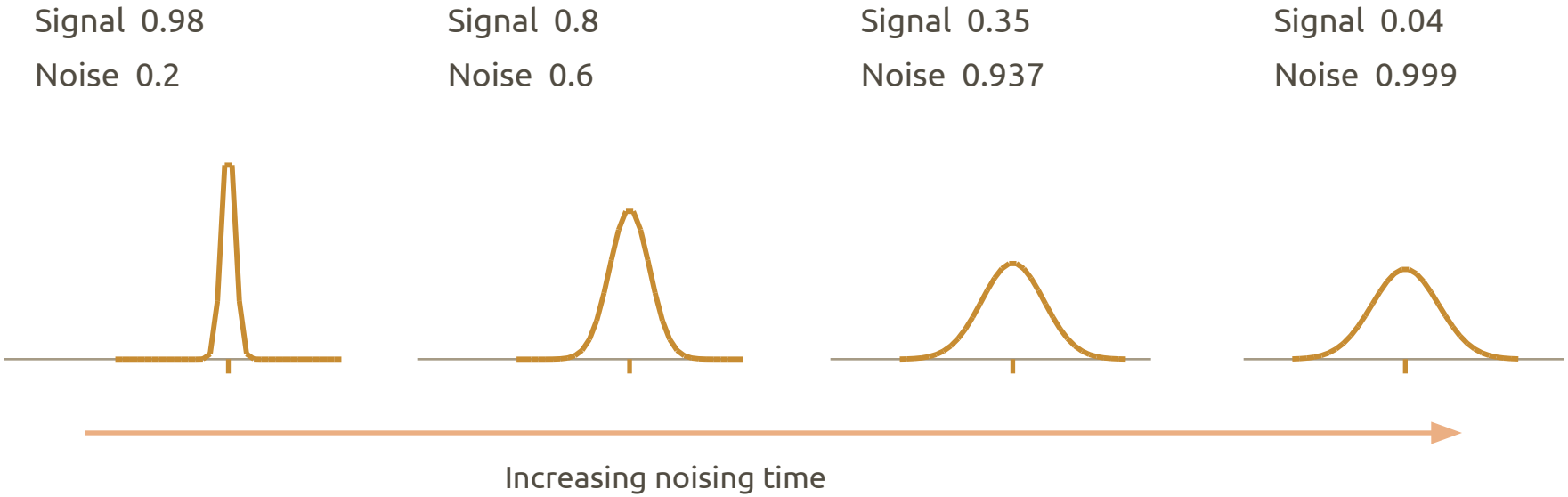


As signal fades, the conditional cloud expands



The signal coefficient moves the center toward zero. The noise standard deviation grows toward one.

As signal fades, the conditional cloud expands



The signal coefficient moves the center toward zero. The noise standard deviation grows toward one. With sufficiently small signal, the terminal marginal is approximately standard Gaussian.

Let’s implement the corruption we just calculated

Let’s implement the corruption we just calculated

Use the signal coefficient and noise standard deviation at the selected time.

Let’s implement the corruption we just calculated

Use the signal coefficient and noise standard deviation at the selected time. Draw a Gaussian tensor and combine it with the clean batch.

Let’s implement the corruption we just calculated

Use the signal coefficient and noise standard deviation at the selected time. Draw a Gaussian tensor and combine it with the clean batch.

From the time input, we select how much signal to retain and how much noise to add.

Let’s look at the forward process in PyTorch

Let’s look at the forward process in PyTorch

The class stores a noise schedule and samples a noisy batch directly.

Each batch item receives its own stored noise index.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

We store T noise levels.

Python indices run from 0 to T – 1. Index 0 already applies the first noising step.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

torch.linspace creates T noise variances between beta_start and beta_end.

Each variance controls the new noise at one step.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Compute the signal retention at each step.

$$\alpha_k = 1 - \beta_k$$

$$\overline{\alpha}_k = \prod_{j=1}^{k} \alpha_j$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Compute the signal retention at each step.

$$\alpha_k = 1 - \beta_k$$

$$\overline{\alpha}_k = \prod_{j=1}^{k} \alpha_j$$

The one-step retention α_k equals one minus the noise variance β_k, and cumulative retention $\bar{\alpha}_k$ multiplies α_j from step one through k.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Compute the signal retention at each step.

$$\alpha_k = 1 - \beta_k$$

$$\overline{\alpha}_k = \prod_{j=1}^{k} \alpha_j$$

The one-step retention α_k equals one minus the noise variance β_k, and cumulative retention $\bar{\alpha}_k$ multiplies α_j from step one through k.

Python index k – 1 stores $\bar{\alpha}_k$.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Precompute the two corruption coefficients.

$$a_k = \sqrt{\overline{\alpha}_k}$$

$$\sigma_k = \sqrt{1 - \overline{\alpha}_k}$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Precompute the two corruption coefficients.

$$a_k = \sqrt{\overline{\alpha}_k}$$

$$\sigma_k = \sqrt{1 - \overline{\alpha}_k}$$

The signal coefficient a_k equals square root $\bar{\alpha}_k$, and the noise standard deviation σ_k equals square root one minus $\bar{\alpha}_k$.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

q_sample uses the Gaussian transition at the selected noise level.

$$X_k \mid X_0 \sim \mathcal{N}(a_k X_0, \sigma_k^2 I)$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

q_sample uses the Gaussian transition at the selected noise level.

$$X_k \mid X_0 \sim \mathcal{N}(a_k X_0, \sigma_k^2 I)$$

The noisy state X_k given the clean state X_0 is Gaussian with mean $a_k X_0$ and covariance σ_k^2 times the identity matrix I.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

torch.randn_like draws noise with the same shape, dtype, and device as x_0.

$$\varepsilon \sim \mathcal{N}(0, I)$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

torch.randn_like draws noise with the same shape, dtype, and device as x_0.

$$\varepsilon \sim \mathcal{N}(0, I)$$

Each noise coordinate is an independent standard Gaussian draw, with mean zero and variance one.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Gather one coefficient per batch item. Add singleton dimensions so it broadcasts across that item.

$$x_k = a_k x_0 + \sigma_k \varepsilon$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Gather one coefficient per batch item. Add singleton dimensions so it broadcasts across that item.

$$x_k = a_k x_0 + \sigma_k \varepsilon$$

The noisy sample x_k equals its signal coefficient a_k times the clean sample x_0, plus its noise scale σ_k times the Gaussian draw ε.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

Let’s look at the forward process in PyTorch

Gather one coefficient per batch item. Add singleton dimensions so it broadcasts across that item.

$$x_k = a_k x_0 + \sigma_k \varepsilon$$

The noisy sample x_k equals its signal coefficient a_k times the clean sample x_0, plus its noise scale σ_k times the Gaussian draw ε.

Return x_t and noise so training retains the exact noise target.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiffusionForwardProcess:
    def __init__(self, timesteps=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):
        self.T = timesteps
        self.device = device

        # Linearly spaced beta schedule
        self.betas = torch.linspace(beta_start, beta_end, self.T).to(device)  # shape: [T]
        self.alphas = 1.0 - self.betas                                        # shape: [T]
        self.alphas_bar = torch.cumprod(self.alphas, dim=0)                   # shape: [T]
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)                    # shape: [T]
        self.sqrt_one_minus_alphas_bar = torch.sqrt(1.0 - self.alphas_bar)    # shape: [T]

    def q_sample(self, x_0, t):
        """
        Sample x_t from q(x_t | x_0) using closed-form.

        Args:
            x_0: original data, shape [batch_size, ...]
            t: timestep tensor, shape [batch_size] with integers in [0, T)

        Returns:
            x_t: noisy version of x_0 at timestep t
            epsilon: the noise added
        """
        batch_size = x_0.shape[0]
        noise = torch.randn_like(x_0)

        # Gather correct sqrt(alpha_bar_t) and sqrt(1 - alpha_bar_t) for each t in the batch
        sqrt_ab = self.sqrt_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))
        sqrt_1mab = self.sqrt_one_minus_alphas_bar[t].reshape(batch_size, *((1,) * (x_0.dim() - 1)))

        x_t = sqrt_ab * x_0 + sqrt_1mab * noise
        return x_t, noise
```

To reverse the noise, we need a direction from the density

To reverse the noise, we need a direction from the density

To reverse the forward process, we need to estimate how the noisy-data density changes near the current state.

To reverse the noise, we need a direction from the density

To reverse the forward process, we need to estimate how the noisy-data density changes near the current state.

We get that local direction from the log-density gradient.

The score points toward increasing log density

The score points toward increasing log density

At a fixed time t, let $p_t(x)$ be the noisy-data density. Differentiate its logarithm with respect to the state x.

$$s_t(x) = \nabla_x \log p_t(x)$$

The score points toward increasing log density

At a fixed time t, let $p_t(x)$ be the noisy-data density. Differentiate its logarithm with respect to the state x.

$$s_t(x) = \nabla_x \log p_t(x)$$

The score $s_t(x)$ equals the gradient with respect to the state x of the log probability density $\log p_t(x)$.

The score points toward increasing log density

At a fixed time t, let $p_t(x)$ be the noisy-data density. Differentiate its logarithm with respect to the state x.

$$s_t(x) = \nabla_x \log p_t(x)$$

The score $s_t(x)$ equals the gradient with respect to the state x of the log probability density $\log p_t(x)$.

Move in the score direction to increase log density locally. At a stationary point of the density, the score is zero.

For a Gaussian, the score pulls toward the mean

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

The Gaussian log density log p(x) equals a constant C minus the squared displacement x minus μ divided by twice the variance v.

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

Differentiate with respect to x. The constant C disappears.

$$s(x) = -\frac{x-\mu}{v}$$

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

Differentiate with respect to x. The constant C disappears.

$$s(x) = -\frac{x-\mu}{v}$$

The Gaussian score s(x) equals the negative displacement from the mean, x minus μ, divided by the variance v.

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

Differentiate with respect to x. The constant C disappears.

$$s(x) = -\frac{x-\mu}{v}$$

The Gaussian score s(x) equals the negative displacement from the mean, x minus μ, divided by the variance v.

For a Gaussian with mean μ = 2 and variance v = 4, evaluating the score at x = 5 gives −0.75.

For a Gaussian, the score pulls toward the mean

Take a Gaussian with mean μ and variance v > 0.

$$p(x) = \mathcal{N}(\mu, v), \qquad \log p(x) = C - \frac{(x-\mu)^2}{2v}$$

Differentiate with respect to x. The constant C disappears.

$$s(x) = -\frac{x-\mu}{v}$$

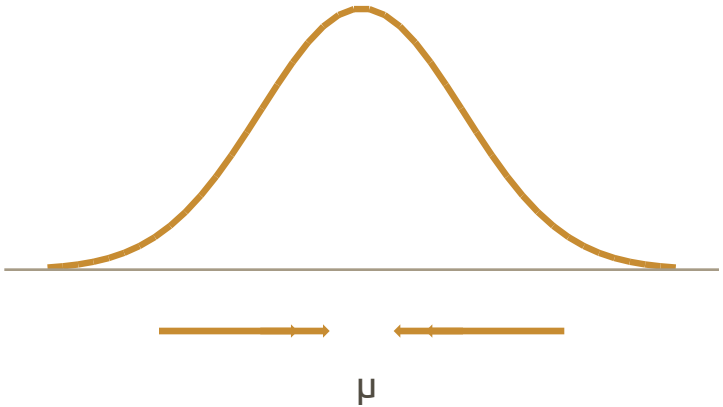
The Gaussian score s(x) equals the negative displacement from the mean, x minus μ, divided by the variance v.

For a Gaussian with mean μ = 2 and variance v = 4, evaluating the score at x = 5 gives −0.75. A wider Gaussian has a smaller score magnitude at the same displacement.

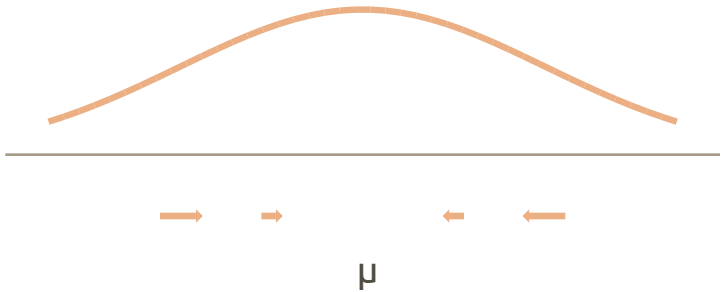
The direction changes on opposite sides of the mean

The direction changes on opposite sides of the mean

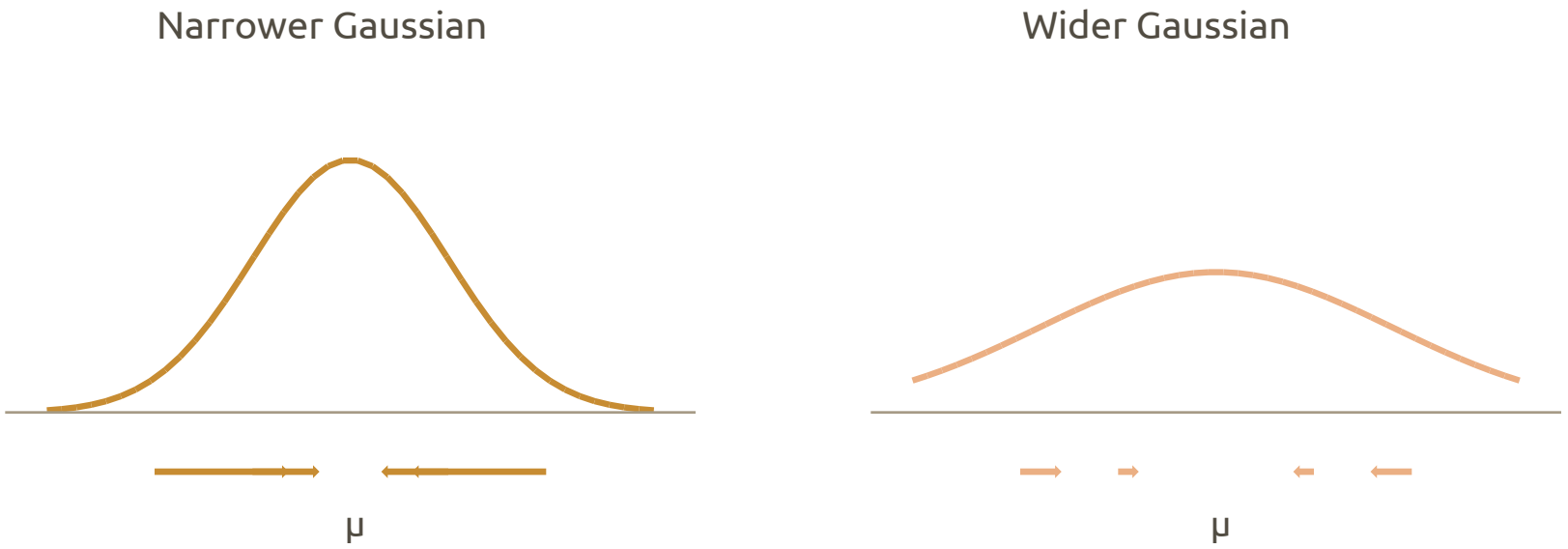
Narrower Gaussian



Wider Gaussian



The direction changes on opposite sides of the mean



To the left of the mean, the score is positive. To the right, it is negative. The arrows become weaker when the variance increases.

The unknown normalizing constant disappears

The unknown normalizing constant disappears

Let E_θ be an energy and Z_θ its normalizing constant.

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

The unknown normalizing constant disappears

Let E_θ be an energy and Z_θ its normalizing constant.

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

The model density $p_\theta(x)$ equals the exponential of negative energy $E_\theta(x)$, divided by the normalizing constant Z_θ.

The unknown normalizing constant disappears

Let E_θ be an energy and Z_θ its normalizing constant.

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

Take the log and differentiate with respect to the state x.

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$$

The unknown normalizing constant disappears

Let E_θ be an energy and Z_θ its normalizing constant.

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

Take the log and differentiate with respect to the state x.

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$$

The model score $\nabla_x \log p_\theta(x)$ equals the negative energy gradient $\nabla_x E_\theta(x)$, because the normalizing constant has no dependence on x.

The unknown normalizing constant disappears

Let E_θ be an energy and Z_θ its normalizing constant.

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

Take the log and differentiate with respect to the state x.

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$$

The model score $\nabla_x \log p_\theta(x)$ equals the negative energy gradient $\nabla_x E_\theta(x)$, because the normalizing constant has no dependence on x.

This cancellation motivated score matching as a way to train models with difficult normalizing constants.

We would like to fit the unknown data score

We would like to fit the unknown data score

Let $s_\theta(x)$ predict the score of the data density p(x). The parameters θ control the prediction.

$$J(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

We would like to fit the unknown data score

Let $s_\theta(x)$ predict the score of the data density p(x). The parameters θ control the prediction.

$$J(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

The score-matching loss J(θ) equals one-half the expected squared distance between the predicted score $s_\theta(x)$ and the data score ∇_x log p(x), averaging over data samples x.

We would like to fit the unknown data score

Let $s_\theta(x)$ predict the score of the data density p(x). The parameters θ control the prediction.

$$J(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

The score-matching loss J(θ) equals one-half the expected squared distance between the predicted score $s_\theta(x)$ and the data score ∇_x log p(x), averaging over data samples x.

We only observe data samples, so we cannot evaluate this target score directly.

We would like to fit the unknown data score

Let $s_\theta(x)$ predict the score of the data density p(x). The parameters θ control the prediction.

$$J(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

The score-matching loss J(θ) equals one-half the expected squared distance between the predicted score $s_\theta(x)$ and the data score ∇_x log p(x), averaging over data samples x.

We only observe data samples, so we cannot evaluate this target score directly. We therefore need to rewrite the loss using quantities we can calculate.

Rewrite the score loss using integration by parts

Rewrite the score loss using integration by parts

Expand the squared error and integrate its cross term by parts, assuming sufficient smoothness and vanishing boundary terms.

$$J(\theta) = \mathbb{E}_p\left[\tfrac{1}{2}\|s_\theta(x)\|^2 + \nabla_x \cdot s_\theta(x)\right] + C$$

Rewrite the score loss using integration by parts

Expand the squared error and integrate its cross term by parts, assuming sufficient smoothness and vanishing boundary terms.

$$J(\theta) = \mathbb{E}_p\left[\tfrac{1}{2}\|s_\theta(x)\|^2 + \nabla_x \cdot s_\theta(x)\right] + C$$

The same loss J(θ) equals the expected sum of one-half the squared score magnitude and the score divergence $\nabla_x \cdot s_\theta(x)$, plus a constant C that does not depend on the model parameters θ.

Rewrite the score loss using integration by parts

Expand the squared error and integrate its cross term by parts, assuming sufficient smoothness and vanishing boundary terms.

$$J(\theta) = \mathbb{E}_p\left[\frac{1}{2}\|s_\theta(x)\|^2 + \nabla_x \cdot s_\theta(x)\right] + C$$

The same loss J(θ) equals the expected sum of one-half the squared score magnitude and the score divergence $\nabla_x \cdot s_\theta(x)$, plus a constant C that does not depend on the model parameters θ.

We can evaluate this loss from data samples and derivatives of the score model.

Rewrite the score loss using integration by parts

Expand the squared error and integrate its cross term by parts, assuming sufficient smoothness and vanishing boundary terms.

$$J(\theta) = \mathbb{E}_p\left[\tfrac{1}{2}\|s_\theta(x)\|^2 + \nabla_x \cdot s_\theta(x)\right] + C$$

The same loss J(θ) equals the expected sum of one-half the squared score magnitude and the score divergence $\nabla_x \cdot s_\theta(x)$, plus a constant C that does not depend on the model parameters θ.

We can evaluate this loss from data samples and derivatives of the score model. In high dimensions, evaluating the divergence can be expensive.

Let’s fit a Gaussian score with one parameter

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

After dropping the parameter-independent constant, the loss is one-half a squared times the expected squared sample, minus a. Since that expectation is one, we get one-half a squared minus a.

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

First evaluate the parameter-dependent loss at coefficient a = 0.5.

$$J(0.5) - C = \tfrac{1}{2}(0.5)^2 - 0.5 = -0.375$$

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

First evaluate the parameter-dependent loss at coefficient a = 0.5.

$$J(0.5) - C = \tfrac{1}{2}(0.5)^2 - 0.5 = -0.375$$

At coefficient one-half, the parameter-dependent loss equals negative 0.375.

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Now evaluate the same loss at coefficient a = 1.

$$J(0.5) - C = -0.375, \qquad J(1) - C = \tfrac{1}{2}(1)^2 - 1 = -0.5$$

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Now evaluate the same loss at coefficient a = 1.

$$J(0.5) - C = -0.375, \qquad J(1) - C = \tfrac{1}{2}(1)^2 - 1 = -0.5$$

At coefficient one, the loss equals negative 0.5, which is lower than its value at one-half.

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Differentiate the loss and set the derivative equal to zero.

$$J(0.5) - C = -0.375, \qquad J(1) - C = -0.5$$

$$J'(a) = a - 1 = 0 \quad \Longrightarrow \quad a^\star = 1$$

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Differentiate the loss and set the derivative equal to zero.

$$J(0.5) - C = -0.375, \qquad J(1) - C = -0.5$$

$$J'(a) = a - 1 = 0 \quad \Longrightarrow \quad a^\star = 1$$

The derivative equals a minus one and vanishes at a = 1, so the loss reaches its minimum there.

Let’s fit a Gaussian score with one parameter

For standard Gaussian data, the expected squared sample is one. Use a score prediction of negative a times x, where the coefficient a sets its inward strength.

$$s_a(x) = -ax, \qquad J(a) - C = \mathbb{E}[\tfrac{1}{2}a^2X^2 - a] = \tfrac{1}{2}a^2 - a$$

Differentiate the loss and set the derivative equal to zero.

$$J(0.5) - C = -0.375, \qquad J(1) - C = -0.5$$

$$J'(a) = a - 1 = 0 \quad \Longrightarrow \quad a^\star = 1$$

The derivative equals a minus one and vanishes at a = 1, so the loss reaches its minimum there.

We recover the standard Gaussian score, negative x, by minimizing a loss based on sample averages and model derivatives.

Estimate the score derivatives along random directions

Estimate the score derivatives along random directions

Choose random directions v with $E[vv^T] = I$. The matrix $\nabla_x s_\theta$ is the Jacobian of the predicted score.

$$J_{\mathrm{SSM}} = \mathbb{E}_{x,v}\left[\frac{1}{2}(v^T s_\theta(x))^2 + v^T(\nabla_x s_\theta(x))v\right],$$

$$\mathbb{E}[vv^T] = I$$

Estimate the score derivatives along random directions

Choose random directions v with $E[vv^T] = I$. The matrix $\nabla_x s_\theta$ is the Jacobian of the predicted score.

$$J_{\mathrm{SSM}} = \mathbb{E}_{x,v}\left[\frac{1}{2}(v^T s_\theta(x))^2 + v^T(\nabla_x s_\theta(x))v\right],$$

$$\mathbb{E}[vv^T] = I$$

The sliced loss J_{SSM} averages one-half the squared score projection $v^T s_\theta(x)$, plus the directional derivative $v^T(\nabla_x s_\theta(x))v$, over data samples x and random directions v.

Estimate the score derivatives along random directions

Choose random directions v with $E[vv^T] = I$. The matrix $\nabla_x s_\theta$ is the Jacobian of the predicted score.

$$J_{\mathrm{SSM}} = \mathbb{E}_{x,v}\left[\frac{1}{2}(v^T s_\theta(x))^2 + v^T(\nabla_x s_\theta(x))v\right],$$

$$\mathbb{E}[vv^T] = I$$

The sliced loss J_{SSM} averages one-half the squared score projection $v^T s_\theta(x)$, plus the directional derivative $v^T(\nabla_x s_\theta(x))v$, over data samples x and random directions v.

We can compute a directional derivative without constructing the full Jacobian.

Estimate the score derivatives along random directions

Choose random directions v with $E[vv^T] = I$. The matrix $\nabla_x s_\theta$ is the Jacobian of the predicted score.

$$J_{\mathrm{SSM}} = \mathbb{E}_{x,v}\left[\frac{1}{2}(v^T s_\theta(x))^2 + v^T(\nabla_x s_\theta(x))v\right],$$

$$\mathbb{E}[vv^T] = I$$

The sliced loss J_{SSM} averages one-half the squared score projection $v^T s_\theta(x)$, plus the directional derivative $v^T(\nabla_x s_\theta(x))v$, over data samples x and random directions v.

We can compute a directional derivative without constructing the full Jacobian. With Gaussian corruption, we can calculate a score target for every noisy training example.

Calculate the score of the Gaussian corruption

Calculate the score of the Gaussian corruption

For a fixed clean sample x_0, the corruption density q_t is Gaussian with mean $a_t x_0$ and variance σ_t^2 in each coordinate.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

Calculate the score of the Gaussian corruption

For a fixed clean sample x_0, the corruption density q_t is Gaussian with mean $a_t x_0$ and variance σ_t^2 in each coordinate.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

The corruption score $\nabla_x \log q_t(x \mid x_0)$ equals the negative displacement x minus $a_t x_0$ divided by the noise variance σ_t^2, which also equals negative sampled noise ε divided by σ_t.

Calculate the score of the Gaussian corruption

For a fixed clean sample x_0, the corruption density q_t is Gaussian with mean $a_t x_0$ and variance σ_t^2 in each coordinate.

$$\nabla_x \log q_t(x \mid x_0) = -\frac{x - a_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}$$

The corruption score $\nabla_x \log q_t(x \mid x_0)$ equals the negative displacement x minus $a_t x_0$ divided by the noise variance σ_t^2, which also equals negative sampled noise ε divided by σ_t.

We know this target for every synthetic training pair because we sampled the clean state and the added noise.

Calculate the score from displacement or sampled noise

Calculate the score from displacement or sampled noise

First calculate the center of the Gaussian corruption for clean sample 10 using signal coefficient 0.8.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

Calculate the score from displacement or sampled noise

First calculate the center of the Gaussian corruption for clean sample 10 using signal coefficient 0.8.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

The conditional mean equals signal coefficient 0.8 times clean sample 10, giving 8.

Calculate the score from displacement or sampled noise

Next compare noisy state 7.4 with that center, then divide the negative displacement by variance 0.36.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

$$x_t - \mu_t = 7.4 - 8 = -0.6$$

$$-\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{-0.6}{0.36} = \frac{5}{3}$$

Calculate the score from displacement or sampled noise

Next compare noisy state 7.4 with that center, then divide the negative displacement by variance 0.36.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

$$x_t - \mu_t = 7.4 - 8 = -0.6$$

$$-\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{-0.6}{0.36} = \frac{5}{3}$$

The noisy state 7.4 lies 0.6 below the conditional mean 8. Negating that displacement and dividing by variance 0.36 gives a score of five-thirds.

Calculate the score from displacement or sampled noise

Finally use sampled noise ε = -1 and noise scale 0.6 to compute the same target directly.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

$$-\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{7.4 - 8}{0.36} = \frac{5}{3}$$

$$-\frac{\varepsilon}{\sigma_t} = -\frac{-1}{0.6} = \frac{5}{3}$$

Calculate the score from displacement or sampled noise

Finally use sampled noise ε = -1 and noise scale 0.6 to compute the same target directly.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

$$-\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{7.4 - 8}{0.36} = \frac{5}{3}$$

$$-\frac{\varepsilon}{\sigma_t} = -\frac{-1}{0.6} = \frac{5}{3}$$

The sampled noise is negative one. Dividing its negative by noise scale 0.6 gives the same score, five-thirds.

Calculate the score from displacement or sampled noise

Finally use sampled noise ε = -1 and noise scale 0.6 to compute the same target directly.

$$\mu_t = a_t x_0 = 0.8(10) = 8$$

$$-\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{7.4 - 8}{0.36} = \frac{5}{3}$$

$$-\frac{\varepsilon}{\sigma_t} = -\frac{-1}{0.6} = \frac{5}{3}$$

The sampled noise is negative one. Dividing its negative by noise scale 0.6 gives the same score, five-thirds.

Both routes return the same conditional score. Its positive sign points from 7.4 back toward conditional mean 8. The marginal score then averages across the clean samples that could produce 7.4.

The marginal score averages the possible clean explanations

The marginal score averages the possible clean explanations

Differentiate the mixture of corruption densities and use Bayes' rule to average over possible clean states X_0.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

The marginal score averages the possible clean explanations

Differentiate the mixture of corruption densities and use Bayes' rule to average over possible clean states X_0.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

The noisy-data score $s_t(x)$ equals the expected corruption score $\nabla_x \log q_t(x \mid X_0)$, averaging over clean states X_0 conditional on observing the noisy state X_t equal to x.

The marginal score averages the possible clean explanations

Differentiate the mixture of corruption densities and use Bayes' rule to average over possible clean states X_0.

$$s_t(x) = \mathbb{E}[\nabla_x \log q_t(x \mid X_0) \mid X_t = x]$$

The noisy-data score $s_t(x)$ equals the expected corruption score $\nabla_x \log q_t(x \mid X_0)$, averaging over clean states X_0 conditional on observing the noisy state X_t equal to x.

We can learn this average from training pairs, even though each pair contains only one clean state.

We can learn that posterior average by denoising

We can learn that posterior average by denoising

Sample a time t, a clean state X_0, and Gaussian noise ε. The positive weight w(t) controls the contribution of each noise level.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

We can learn that posterior average by denoising

Sample a time t, a clean state X_0, and Gaussian noise ε. The positive weight w(t) controls the contribution of each noise level.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t) \left\| s_\theta(X_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

The denoising loss L_{DSM} averages the weight w(t) times the squared distance between the predicted score $s_\theta(X_t, t)$ and the target negative ε divided by σ_t, over times t, clean samples X_0, and Gaussian noise ε.

We can learn that posterior average by denoising

Sample a time t, a clean state X_0, and Gaussian noise ε. The positive weight w(t) controls the contribution of each noise level.

$$\mathcal{L}_{\mathrm{DSM}} = \mathbb{E}_{t,X_0,\varepsilon}\left[w(t)\left\|s_\theta(X_t,t)+\frac{\varepsilon}{\sigma_t}\right\|^2\right]$$

The denoising loss L_{DSM} averages the weight w(t) times the squared distance between the predicted score $s_\theta(X_t, t)$ and the target negative ε divided by σ_t, over times t, clean samples X_0, and Gaussian noise ε.

By minimizing squared error, we learn the target's conditional mean, which equals the marginal score.

Noise prediction is a rescaled score prediction

Noise prediction is a rescaled score prediction

Let ε_θ predict the noise. Divide by the known noise scale σ_t.

$$s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}$$

Noise prediction is a rescaled score prediction

Let ε_θ predict the noise. Divide by the known noise scale σ_t.

$$s_\theta(x, t) = -\frac{\varepsilon_\theta(x, t)}{\sigma_t}$$

The predicted score $s_\theta(x, t)$ equals negative predicted noise $\varepsilon_\theta(x, t)$ divided by the noise standard deviation σ_t.

Noise prediction is a rescaled score prediction

Let ε_θ predict the noise. Divide by the known noise scale σ_t.

$$s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}$$

Choose the score-loss weight $w(t) = \sigma_t^2$. Substitution cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\left\|\varepsilon_\theta(X_t,t) - \varepsilon\right\|^2\right]$$

Noise prediction is a rescaled score prediction

Let ε_θ predict the noise. Divide by the known noise scale σ_t.

$$s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}$$

Choose the score-loss weight $w(t) = \sigma_t^2$. Substitution cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\|\varepsilon_\theta(X_t,t) - \varepsilon\|^2\right]$$

The loss L equals the expected squared difference between predicted noise $\varepsilon_\theta(X_t, t)$ and the Gaussian noise ε used to make the training sample.

Noise prediction is a rescaled score prediction

Let ε_θ predict the noise. Divide by the known noise scale σ_t.

$$s_\theta(x,t) = -\frac{\varepsilon_\theta(x,t)}{\sigma_t}$$

Choose the score-loss weight $w(t) = \sigma_t^2$. Substitution cancels the noise-scale factors.

$$\mathcal{L} = \mathbb{E}\left[\|\varepsilon_\theta(X_t,t) - \varepsilon\|^2\right]$$

The loss L equals the expected squared difference between predicted noise $\varepsilon_\theta(X_t, t)$ and the Gaussian noise ε used to make the training sample.

The model predicts the average noise consistent with the observation. Individual corruptions can remain ambiguous.

The same output also estimates the clean state

The same output also estimates the clean state

For a nonzero signal coefficient a_t, rearrange the corruption equation. Use the network's predicted noise in place of the unknown noise.

$$\widehat{x}_0 = \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{a_t} = \frac{x_t + \sigma_t^2 s_\theta(x_t, t)}{a_t}$$

The same output also estimates the clean state

For a nonzero signal coefficient a_t, rearrange the corruption equation. Use the network's predicted noise in place of the unknown noise.

$$\widehat{x_0} = \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{a_t} = \frac{x_t + \sigma_t^2 s_\theta(x_t, t)}{a_t}$$

The clean-state estimate $\hat{x}_0$ equals the noisy state x_t minus the noise scale σ_t times the predicted noise ε_θ, all divided by the signal coefficient a_t, which also equals x_t plus $\sigma_t^2 s_\theta$ divided by a_t.

The same output also estimates the clean state

For a nonzero signal coefficient a_t, rearrange the corruption equation. Use the network's predicted noise in place of the unknown noise.

$$\widehat{x}_0 = \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{a_t} = \frac{x_t + \sigma_t^2 s_\theta(x_t, t)}{a_t}$$

The clean-state estimate $\hat{x}_0$ equals the noisy state x_t minus the noise scale σ_t times the predicted noise ε_θ, all divided by the signal coefficient a_t, which also equals x_t plus $\sigma_t^2 s_\theta$ divided by a_t.

Noise, score, and clean-state prediction carry the same information at a fixed interior noise level.

The noise estimate determines the reconstruction

The noise estimate determines the reconstruction

Suppose the network predicts noise −0.8 for our sample. The Gaussian noise we actually added was −1.

$$\widehat{s}_t = -\frac{-0.8}{0.6} = 1.333,$$

$$\widehat{x}_0 = \frac{7.4 - (0.6)(-0.8)}{0.8} = 9.85$$

The noise estimate determines the reconstruction

Suppose the network predicts noise −0.8 for our sample. The Gaussian noise we actually added was −1.

$$\widehat{s_t} = -\frac{-0.8}{0.6} = 1.333,$$

$$\widehat{x_0} = \frac{7.4 - (0.6)(-0.8)}{0.8} = 9.85$$

The predicted score $\hat{s}_t$ equals negative predicted noise divided by 0.6, giving about 1.333, and the clean estimate $\hat{x}_0$ equals the sum of 7.4 and 0.48, all divided by 0.8, giving 9.85.

The noise estimate determines the reconstruction

Suppose the network predicts noise −0.8 for our sample. The Gaussian noise we actually added was −1.

$$\widehat{s_t} = -\frac{-0.8}{0.6} = 1.333,$$

$$\widehat{x_0} = \frac{7.4 - (0.6)(-0.8)}{0.8} = 9.85$$

The predicted score $\hat{s}_t$ equals negative predicted noise divided by 0.6, giving about 1.333, and the clean estimate $\hat{x}_0$ equals the sum of 7.4 and 0.48, all divided by 0.8, giving 9.85.

The estimate 9.85 is close to this example’s clean value ten.

The noise estimate determines the reconstruction

Suppose the network predicts noise −0.8 for our sample. The Gaussian noise we actually added was −1.

$$\widehat{s_t} = -\frac{-0.8}{0.6} = 1.333,$$

$$\widehat{x_0} = \frac{7.4 - (0.6)(-0.8)}{0.8} = 9.85$$

The predicted score $\hat{s}_t$ equals negative predicted noise divided by 0.6, giving about 1.333, and the clean estimate $\hat{x}_0$ equals the sum of 7.4 and 0.48, all divided by 0.8, giving 9.85.

The estimate 9.85 is close to this example's clean value ten. Even an optimal predictor averages over multiple clean states compatible with the noisy input.